Announcements, August 31, 2026

- All recorded lectures and lecture notes on the USC Brightspace, see https://brightspace.usc.edu/
- We will have 2 guest lecturers this semester (see revised syllabus):
  - September 14, 2026: Dr. Hieu Nguyen of eBay on key-value stores, graph databases, and dataflow execution paradigms for AI workloads.
  - October 19, 2026: Dr. Haoyu Huang of DataBricks on separation of storage from processing and disaggregated DBMSs for AI agents.
- TAs & Office Hours:



Hamed Alimohammadzadeh

halimoha@usc.edu
Monday 12 pm - 4 pm
Room: GCS SB6



Raja Kumar

kumarraj@usc.edu
Wed 9-11 AM SB6
Wed 1-3 PM SB5



Shuqin Zhu

shuqinzh@usc.edu
Fridays 1-5 pm, Room SB8, GCS

Shahram Ghandeharizadeh Office Hours

Office: GCS 505E

Email: shahram@usc.edu

Phone: 213-740-4781

Office hours: Mon 10-12:30 pm, Lower Level 2 of GCS, SB7

Mon 2-4 pm, Lower Level 2 of GCS, Room SB7

By appointment (send me an email)

Gamma

by DeWitt et. al.

Shahram Ghandeharizadeh
CSCI 585

The Gamma Database Machine Project

Assigned Reading

DAVID J. DEWITT, SHAHRAM GHANDEHARIZADEH, DONOVAN A. SCHNEIDER, ALLAN BRICKER, HUI-I HSIAO, AND RICK RASMUSSEN

Abstract—**This paper describes the design of the Gamma database machine and the techniques employed in its implementation. Gamma is a relational database machine currently operating on an Intel iPSC/2 hypercube with 32 processors and 32 disk drives. Gamma employs three key technical ideas which enable the architecture to be scaled to hundreds of processors. First, all relations are horizontally partitioned across multiple disk drives enabling relations to be scanned in parallel. Second, novel parallel algorithms based on hashing are used to implement the complex relational operators such as join and aggregate functions. Third, dataflow scheduling techniques are used to coordinate multioperator queries. By using these techniques it is possible to control the execution of very complex queries with minimal coordination—a necessity for configurations involving a very large number of processors.**

In addition to describing the design of the Gamma software, a thorough performance evaluation of the iPSC/2 hypercube version of Gamma is also presented. In addition to measuring the effect of relation size and indexes on the response time for selection, join, aggregation, and update queries, we also analyze the performance of Gamma relative to the number of processors employed when the sizes of the input relations are kept constant (speedup) and when the sizes of the input relations are increased proportionally to the number of processors (scaleup). The speedup results obtained for both selection and join queries are linear; thus, doubling the number of processors halves the response time for a query. The scaleup results obtained are also quite encouraging. They reveal that a nearly constant response time can be maintained for both selection and join queries as the workload is increased by adding a proportional number of processors and disks.

Index Terms—**Database machines, dataflow query processing, distributed database systems, parallel algorithms, relational database systems.**

I. INTRODUCTION

FOR the last 5 years, the Gamma database machine project has focused on issues associated with the design and implementation of highly parallel database machines. In a number of ways, the design of Gamma is shared memory and centralized control for the execution of its parallel algorithms [3].

As a solution to the problems encountered with DIRECT, Gamma employs what appear today to be relatively straightforward solutions. Architecturally, Gamma is based on a shared-nothing [37] architecture consisting of a number of processors interconnected by a communications network such as a hypercube or a ring, with disks directly connected to the individual processors. It is generally accepted that such architectures can be scaled to incorporate thousands of processors. In fact, Teradata database machines [40] incorporating a shared-nothing architecture with over 200 processors are already in use. The second key idea employed by Gamma is the use of hash-based parallel algorithms. Unlike the algorithms employed by DIRECT, these algorithms require no centralized control and can thus, like the hardware architecture, be scaled almost indefinitely. Finally, to make the best of the limited I/O bandwidth provided by the current generation of disk drives, Gamma employs the concept of *horizontal partitioning* [33] (also termed *declustering* [29]) to distribute the tuples of a relation among multiple disk drives. This design enables large relations to be processed by multiple processors concurrently without incurring any communications overhead.

After the design of the Gamma software was completed in the fall of 1984, work began on the first prototype which was operational by the fall of 1985. This version of Gamma was implemented on top of an existing multicomputer consisting of 20 VAX 11/750 processors [12]. In the period of 1986–1988, the prototype was enhanced through the addition of a number of new operators (e.g., aggregate and update operators), new parallel join meth-

Why is Gamma Relevant to Today's Systems

Gamma was designed for the likes of Facebook, Amazon, eBay, Azure, and other giants of the Internet before they existed.

Gamma hides parallelism using SQL. The user who writes the SQL query is not aware that it is being executed in a parallel manner using many computers. Today's comparable systems include Amazon RedShift and Google BigQuery.

ACM Software Systems Award

Awarded to an institution or individual(s) recognized for developing a software system that has had a lasting influence, reflected in contributions to concepts, in commercial acceptance, or both.

Gamma is the recipient of the ACM Software System Award 2008.

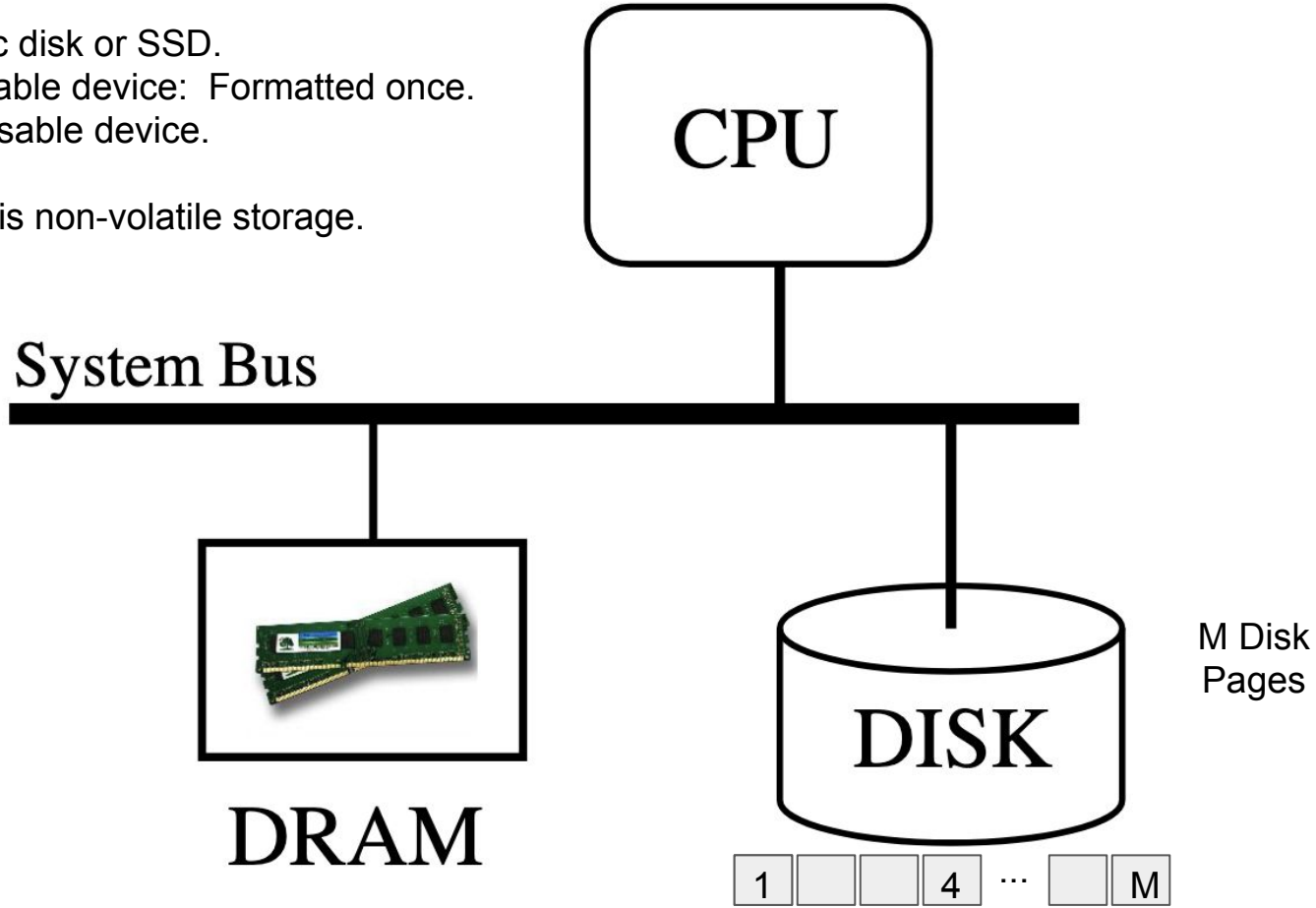
Gamma continues to influence systems designed today.

Node Hardware & Disk I/O

Hardware Organization

Disk may be a magnetic disk or SSD.
Disk is a block addressable device: Formatted once.
DRAM is a byte-addressable device.

DRAM is volatile. Disk is non-volatile storage.

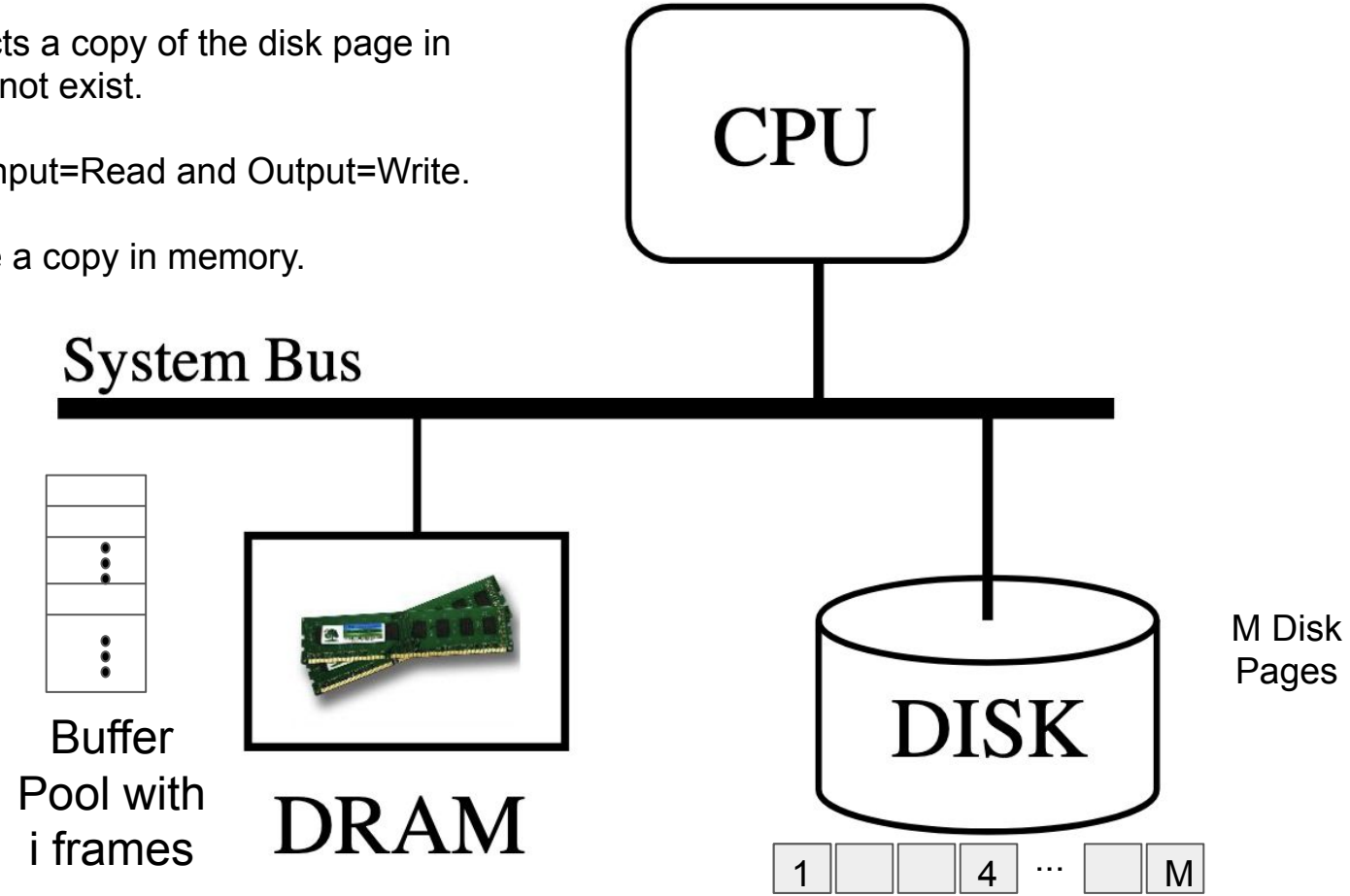


Disk Input/Output (I/O)

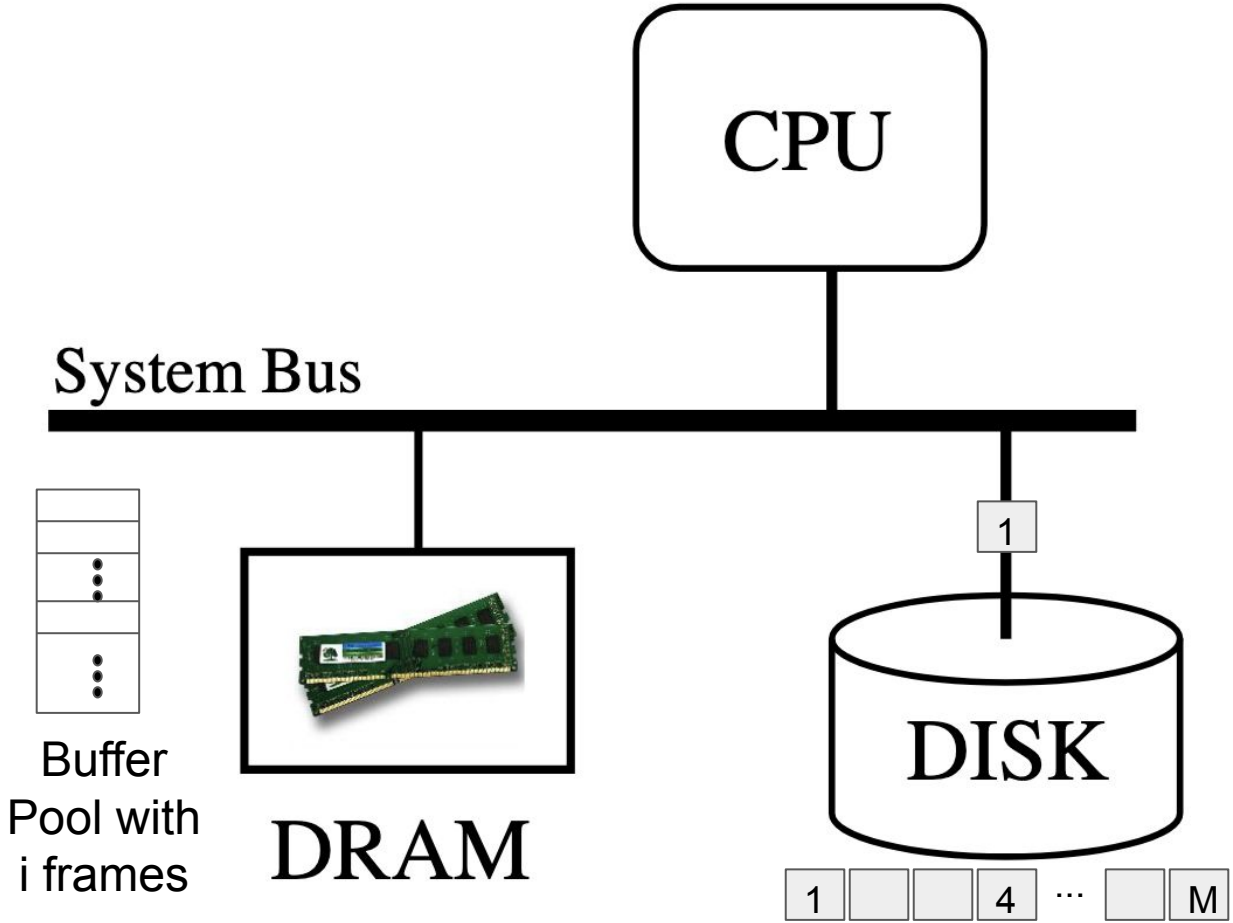
A Disk read constructs a copy of the disk page in memory if one does not exist.

Disk I/O stands for Input=Read and Output=Write.

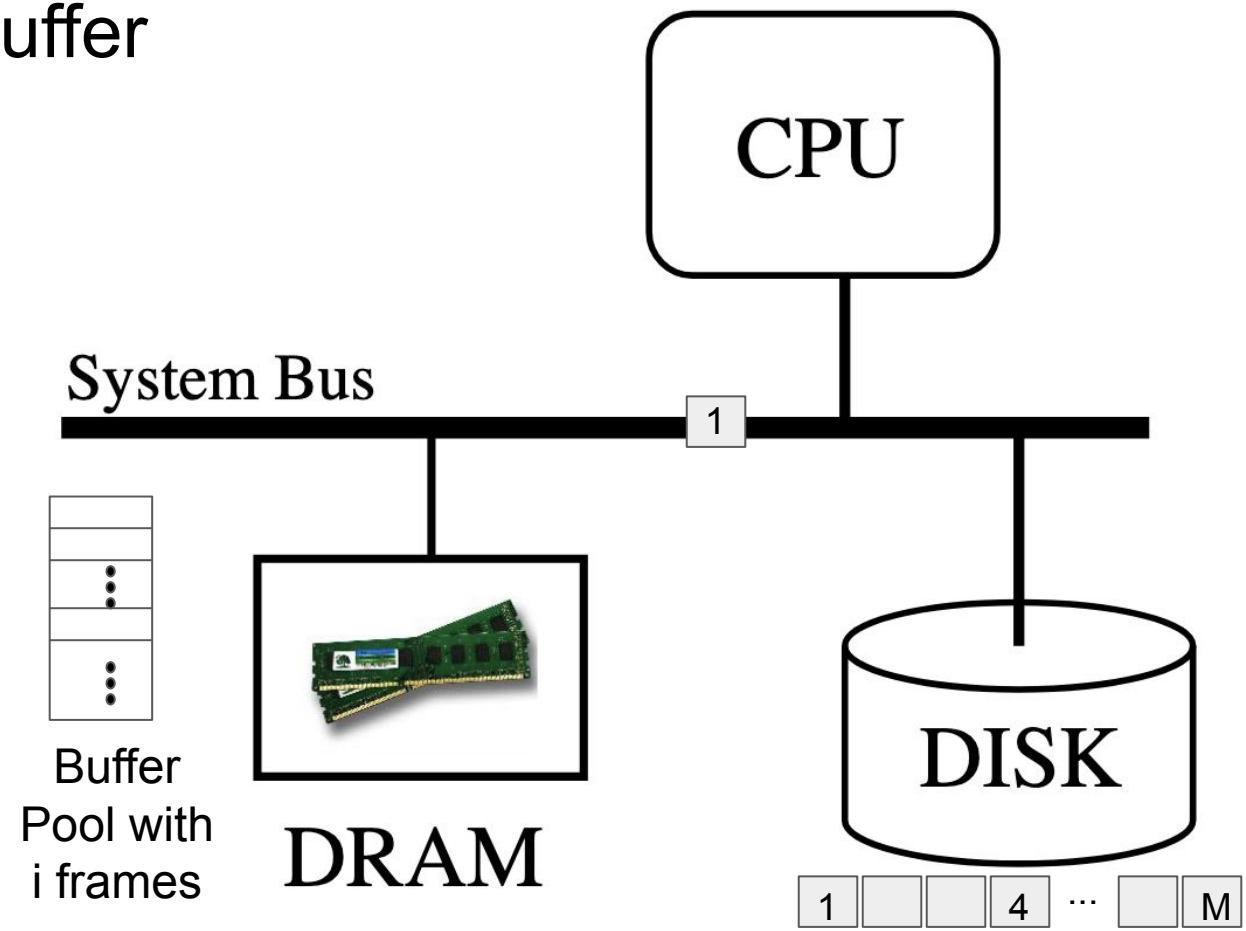
Disk write may leave a copy in memory.



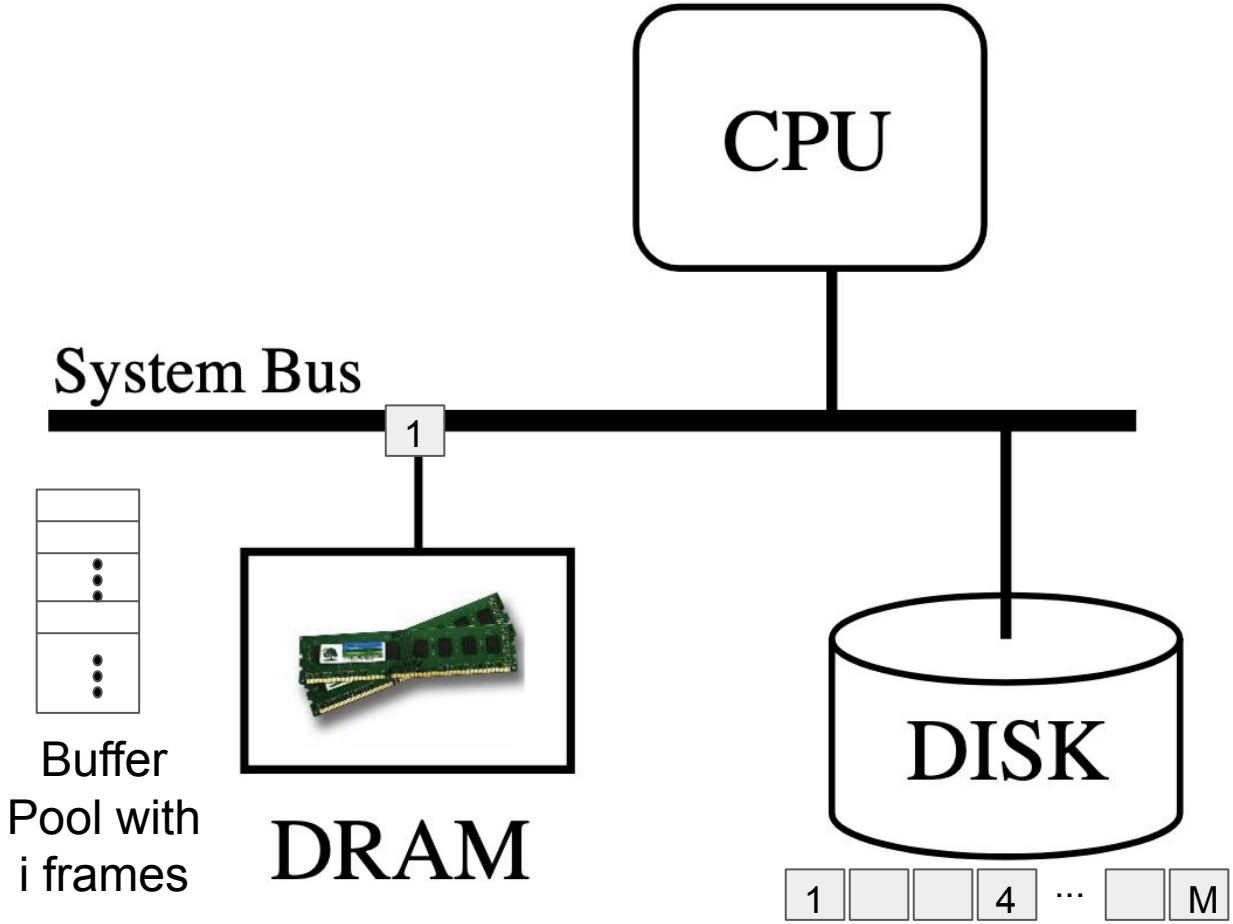
Disk Input/Output (I/O): Read Disk Page 1



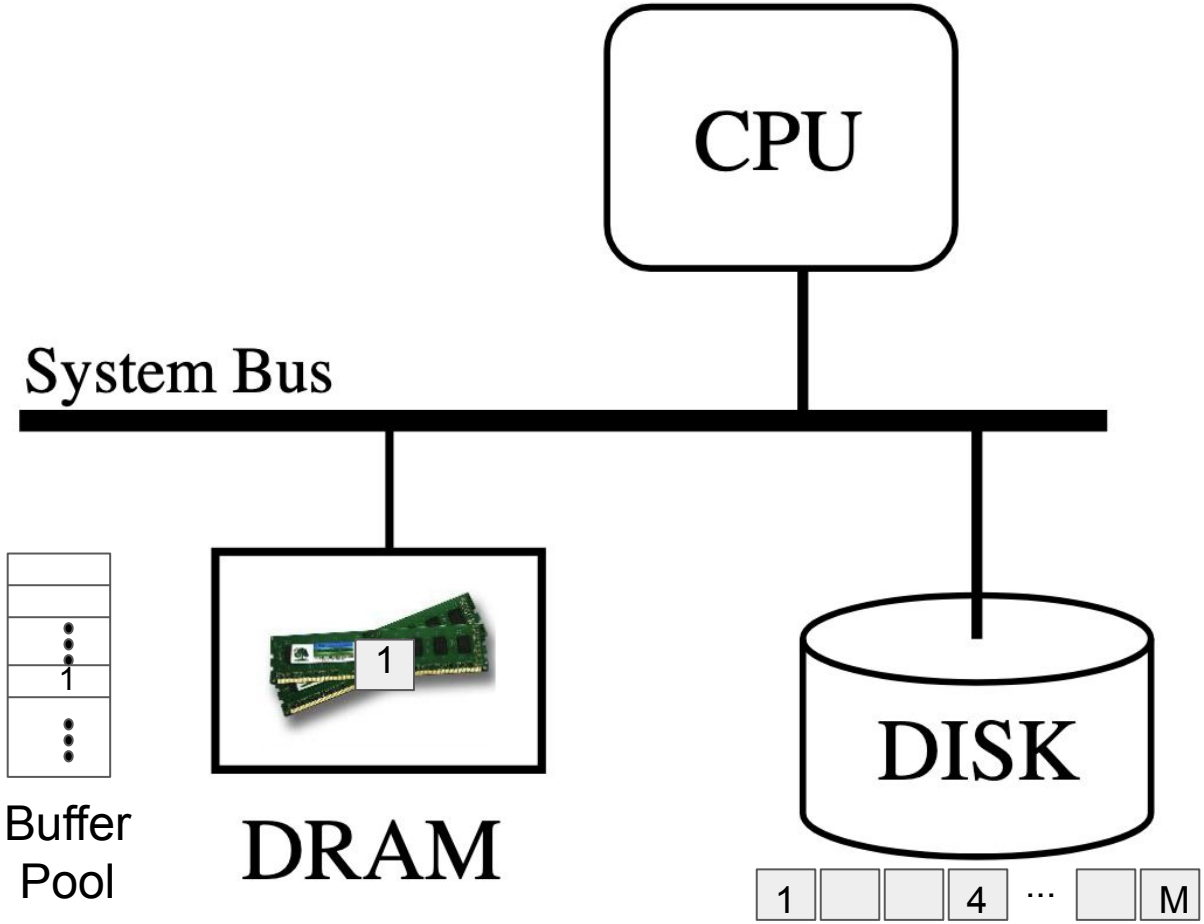
Disk Input/Output (I/O): Page 1 is Written into a Memory Buffer



Disk Input/Output (I/O)

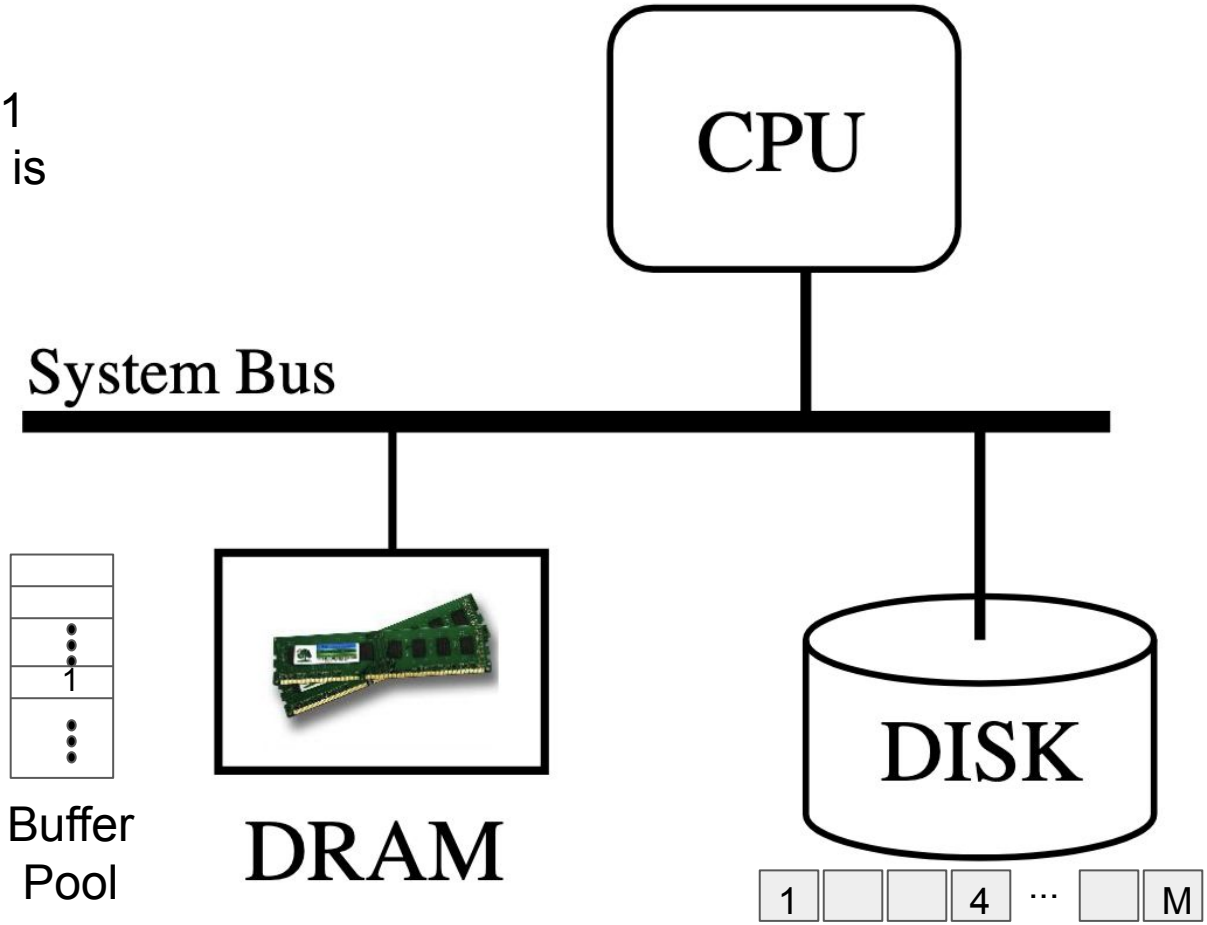


Buffer Pool Consists of Disk Page Frames

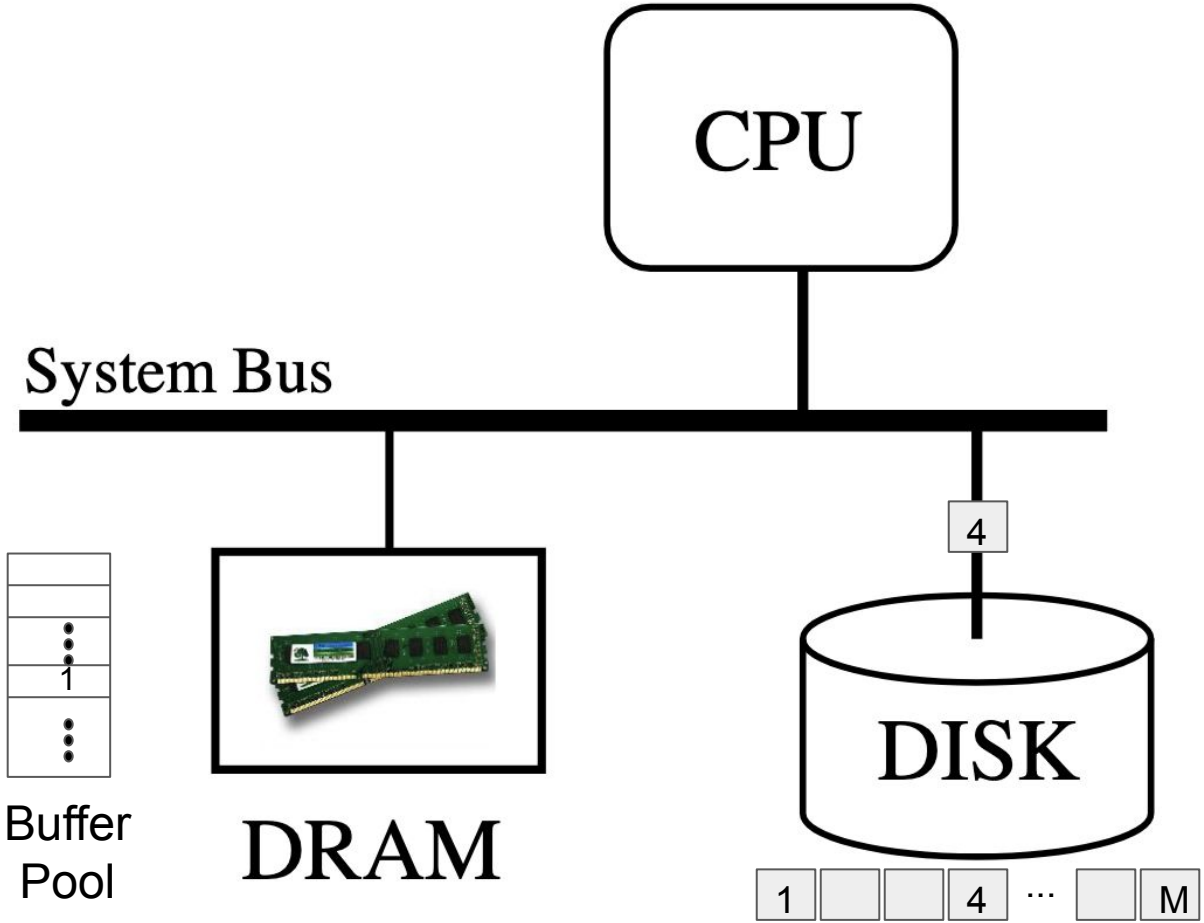


A Disk Page Occupies a Frame

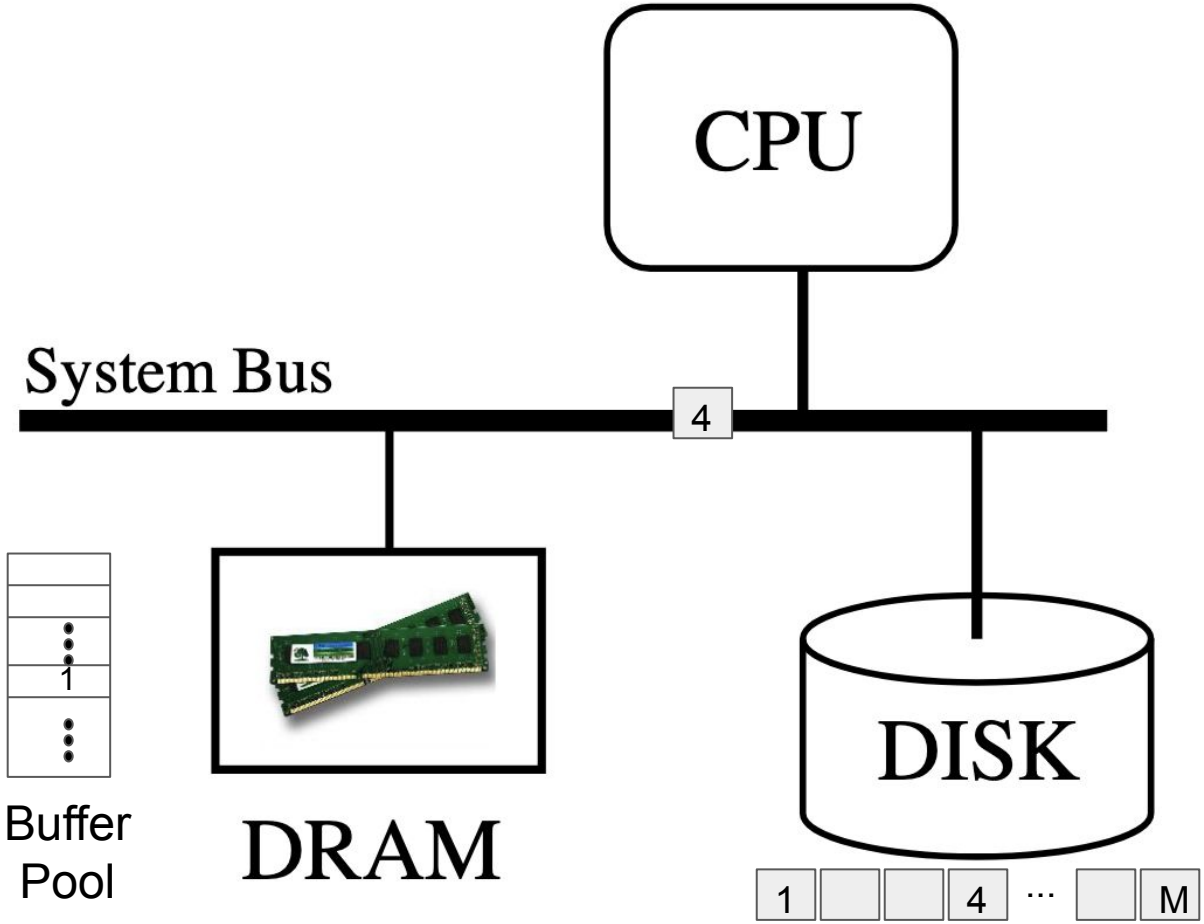
How to find Page 1 in memory once it is referenced?



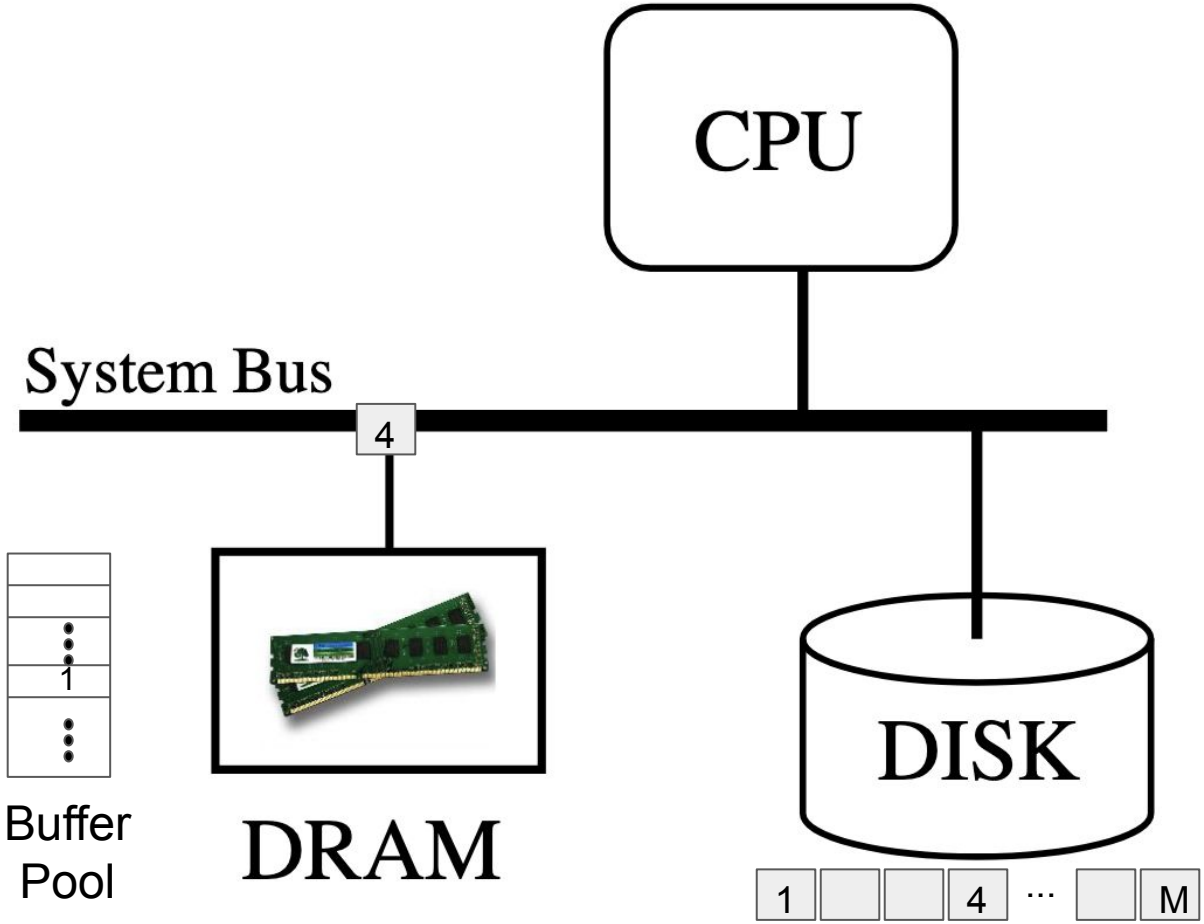
Disk Input/Output (I/O): Read Page 4



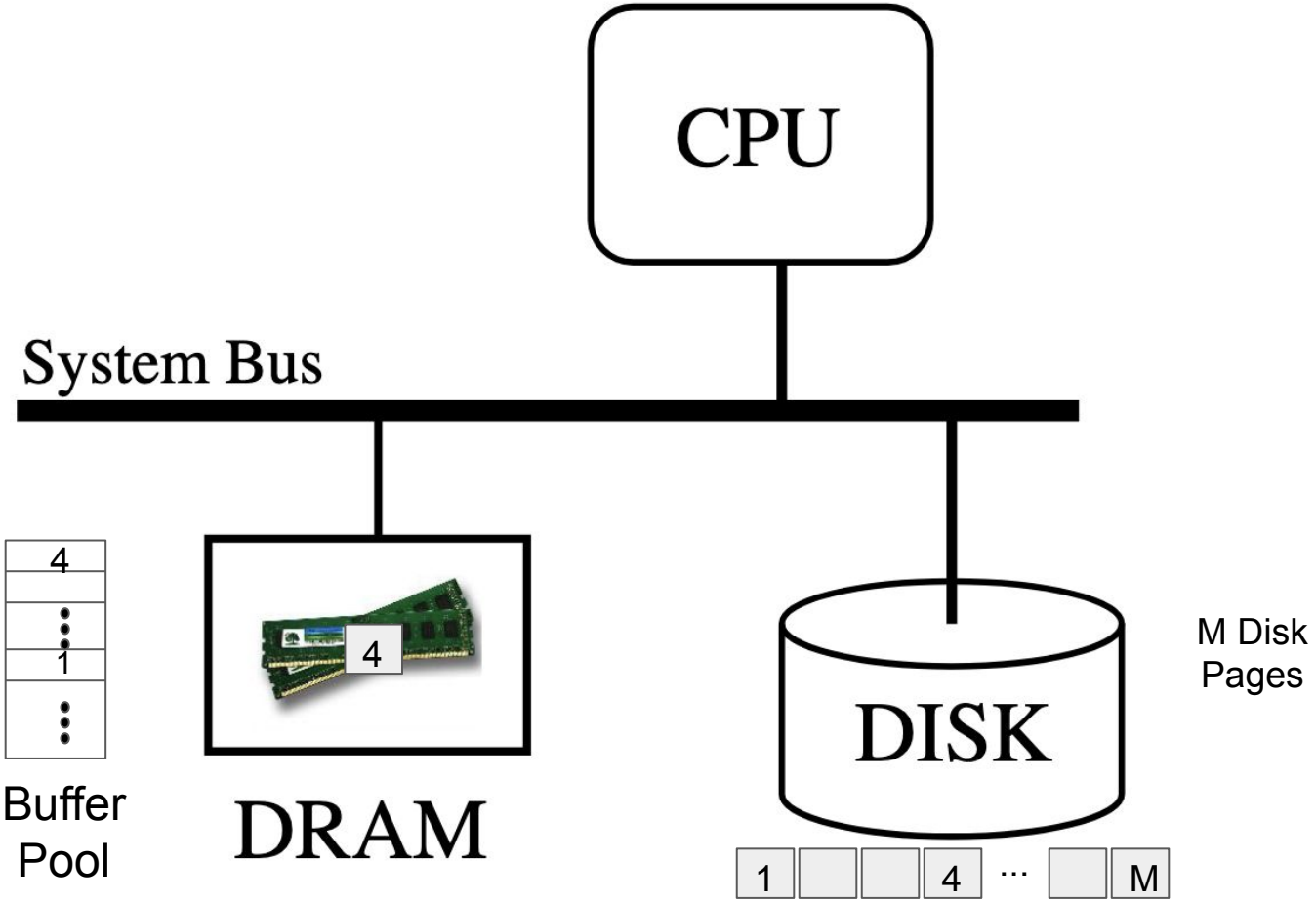
Disk Input/Output (I/O)



Disk Input/Output (I/O)



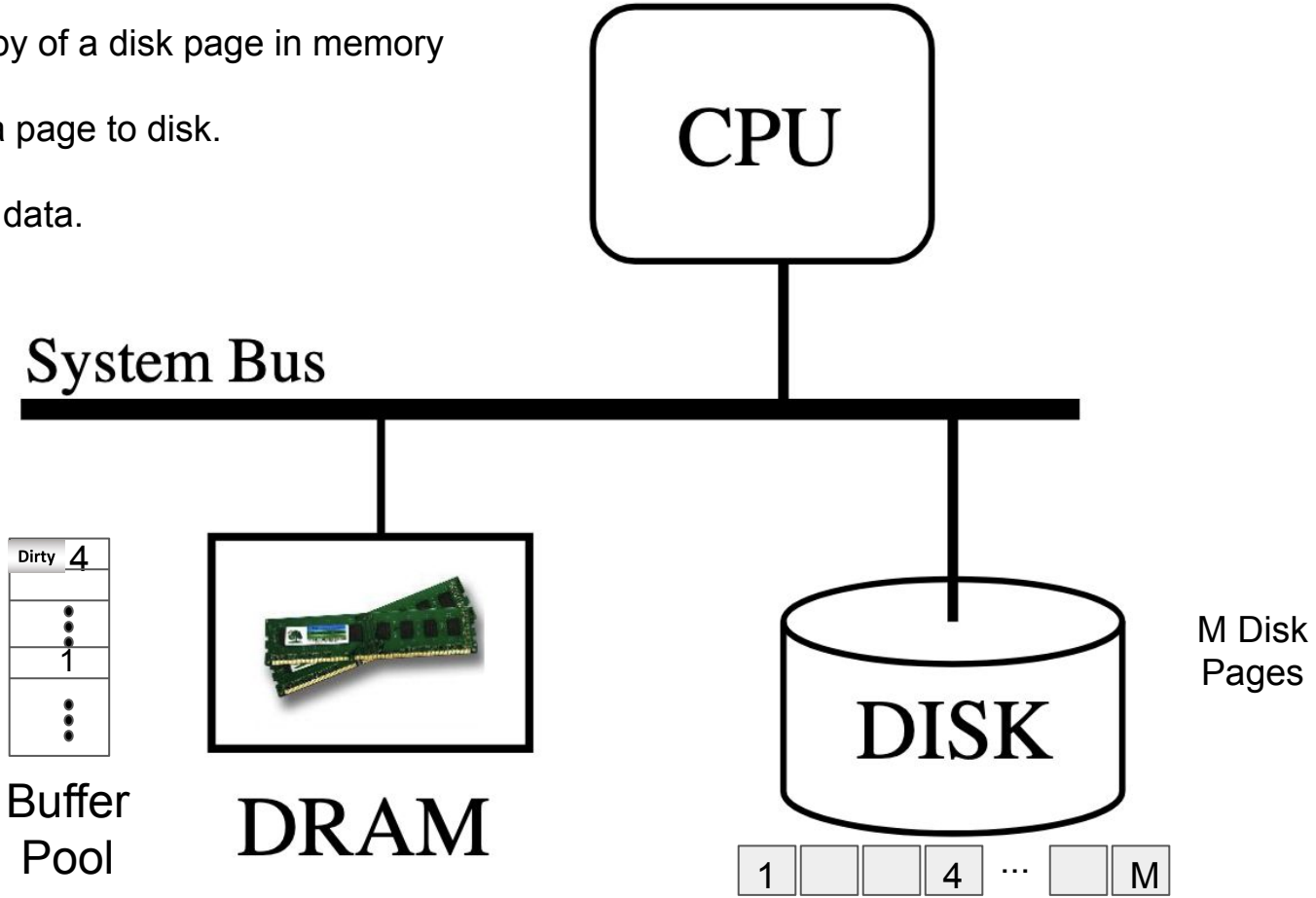
Each Disk Page Occupies a Buffer Pool Frame



Disk Input/Output (I/O)

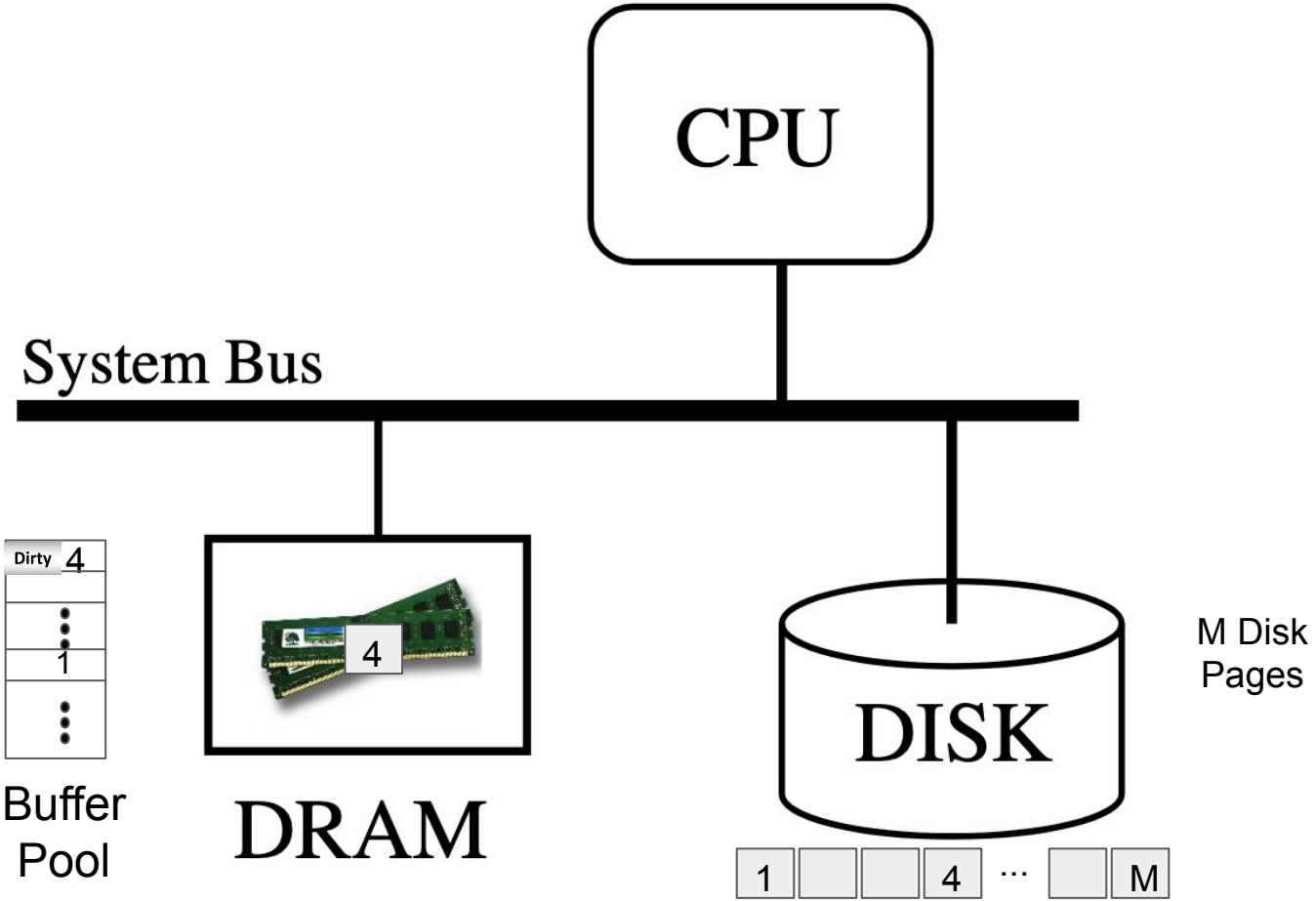
A write updates the copy of a disk page in memory and marks it dirty.
The DBMS may flush a page to disk.

Write Page 4 with new data.

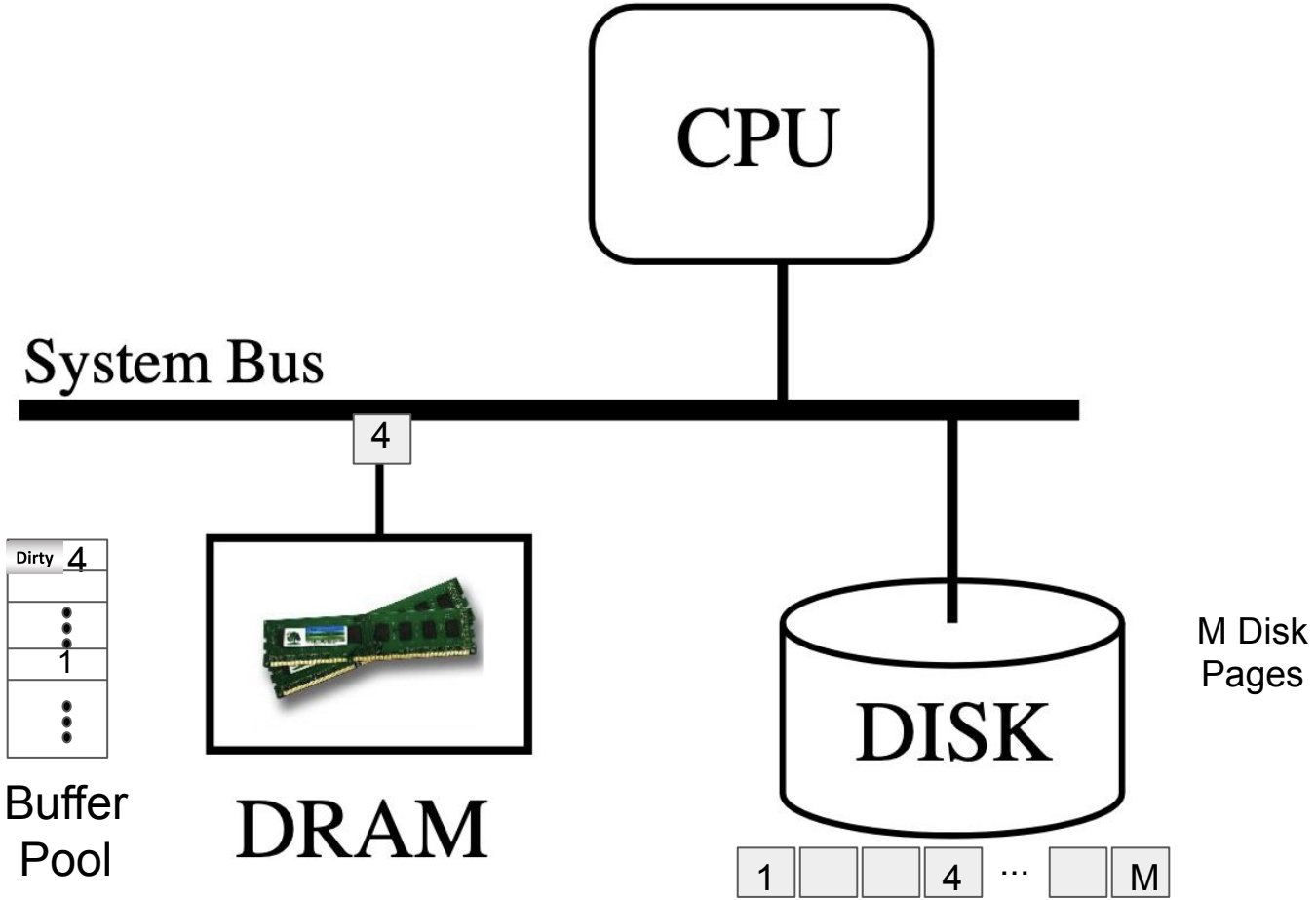


Disk Input/Output (I/O)

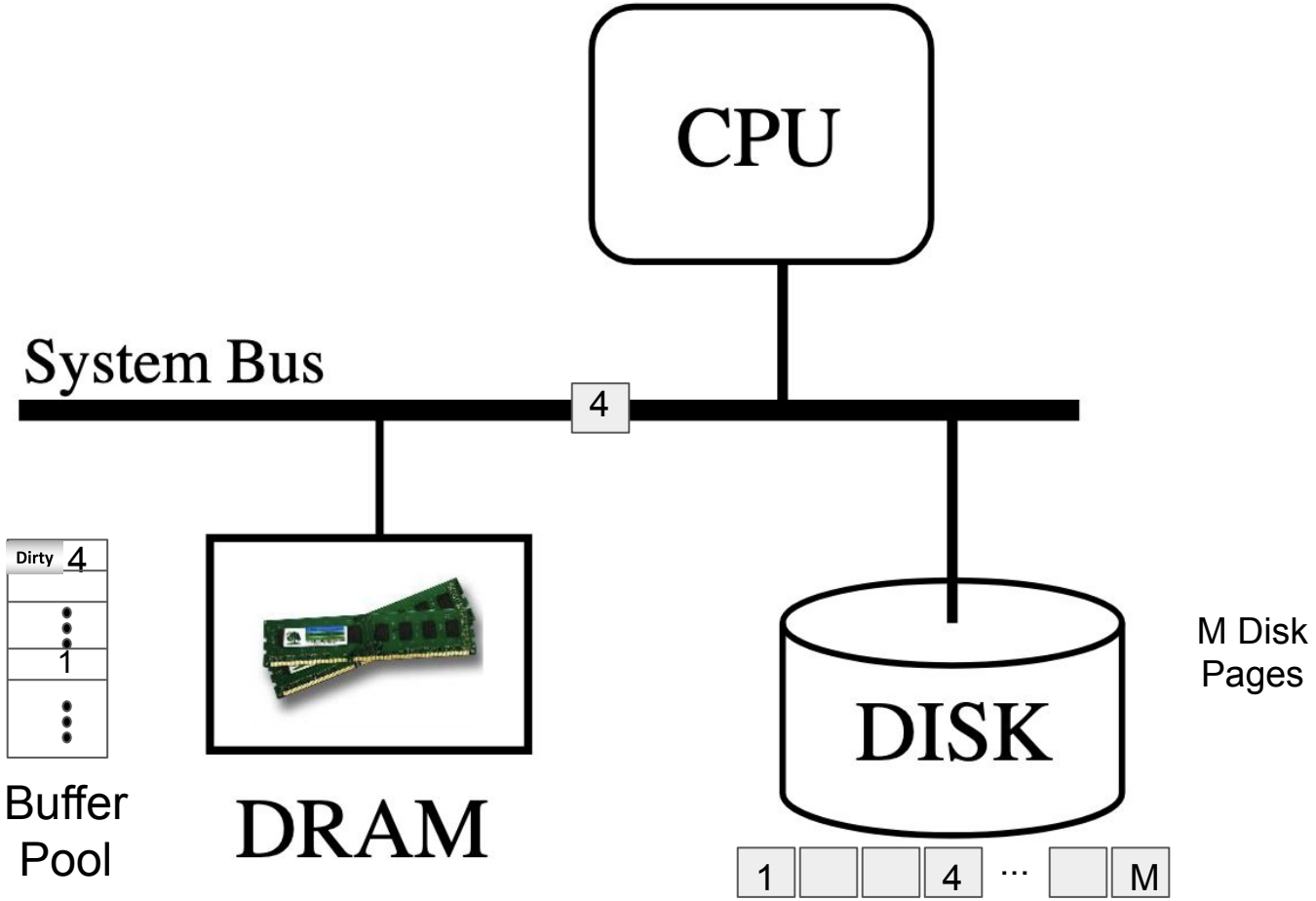
Flush Page 4 to Disk.



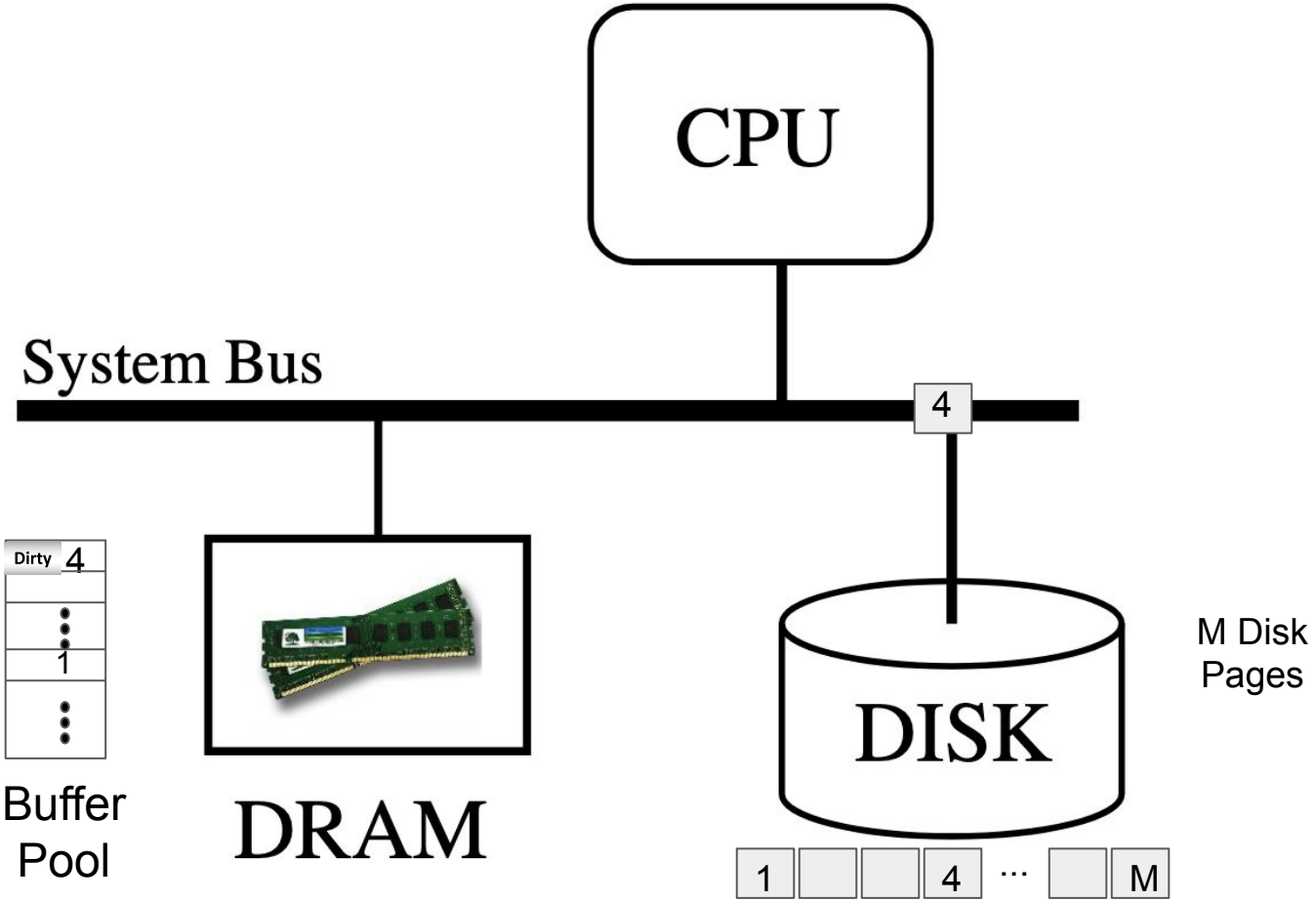
Disk Input/Output (I/O): Flush Page 4 to Disk



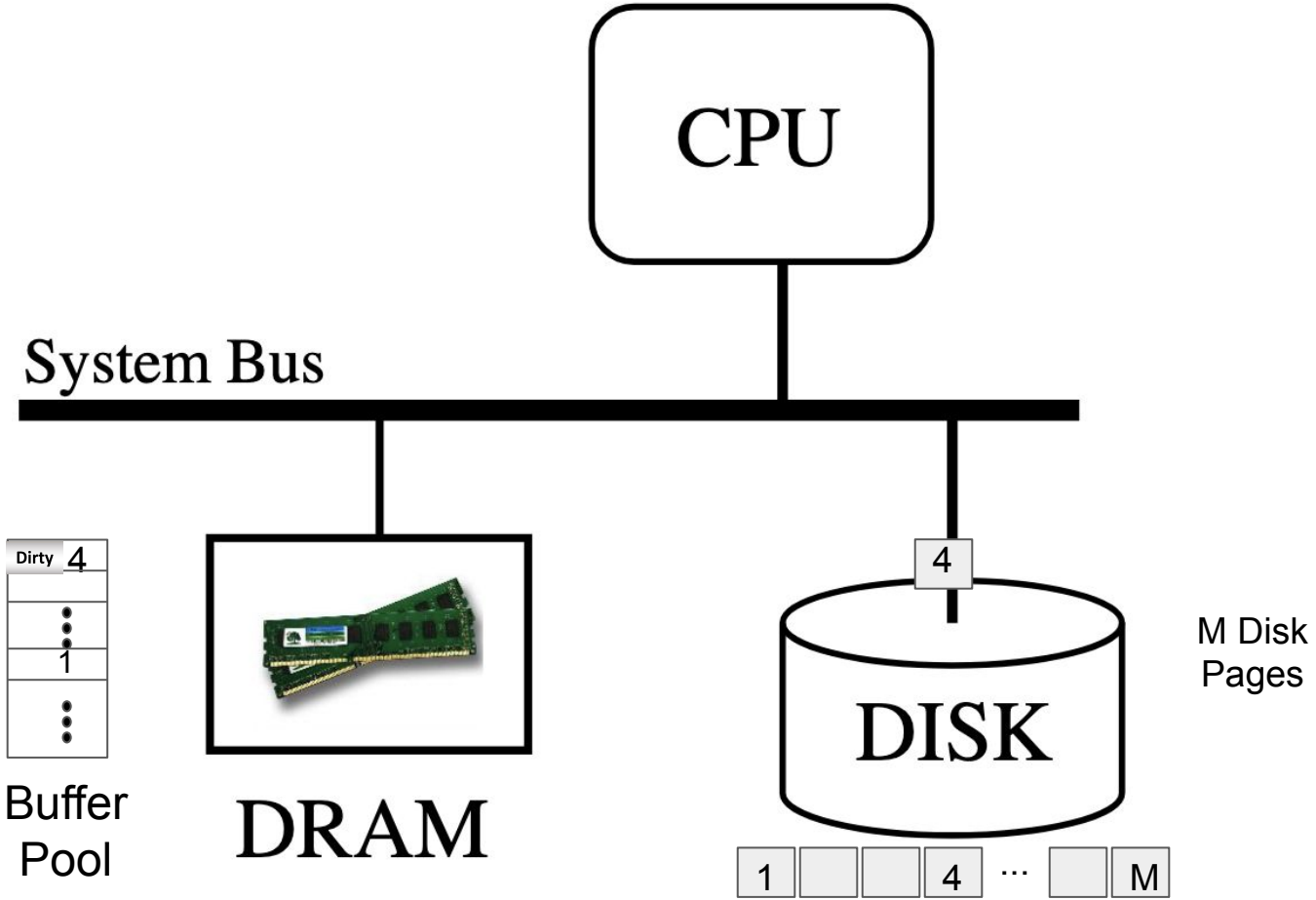
Disk Input/Output (I/O): Flush Page 4 to Disk



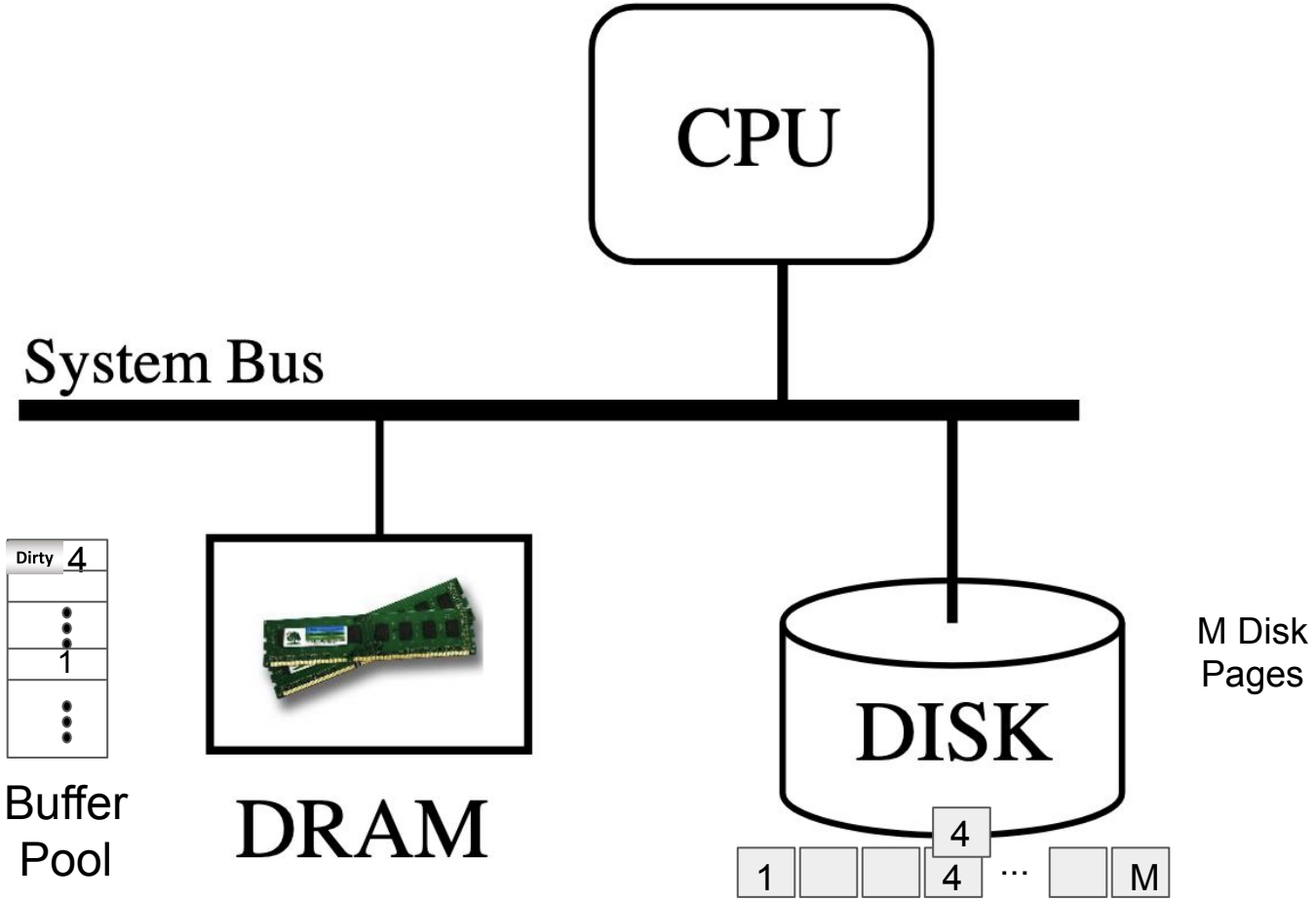
Disk Input/Output (I/O): Flush Page 4 to Disk



Disk Input/Output (I/O): Flush Page 4 to Disk

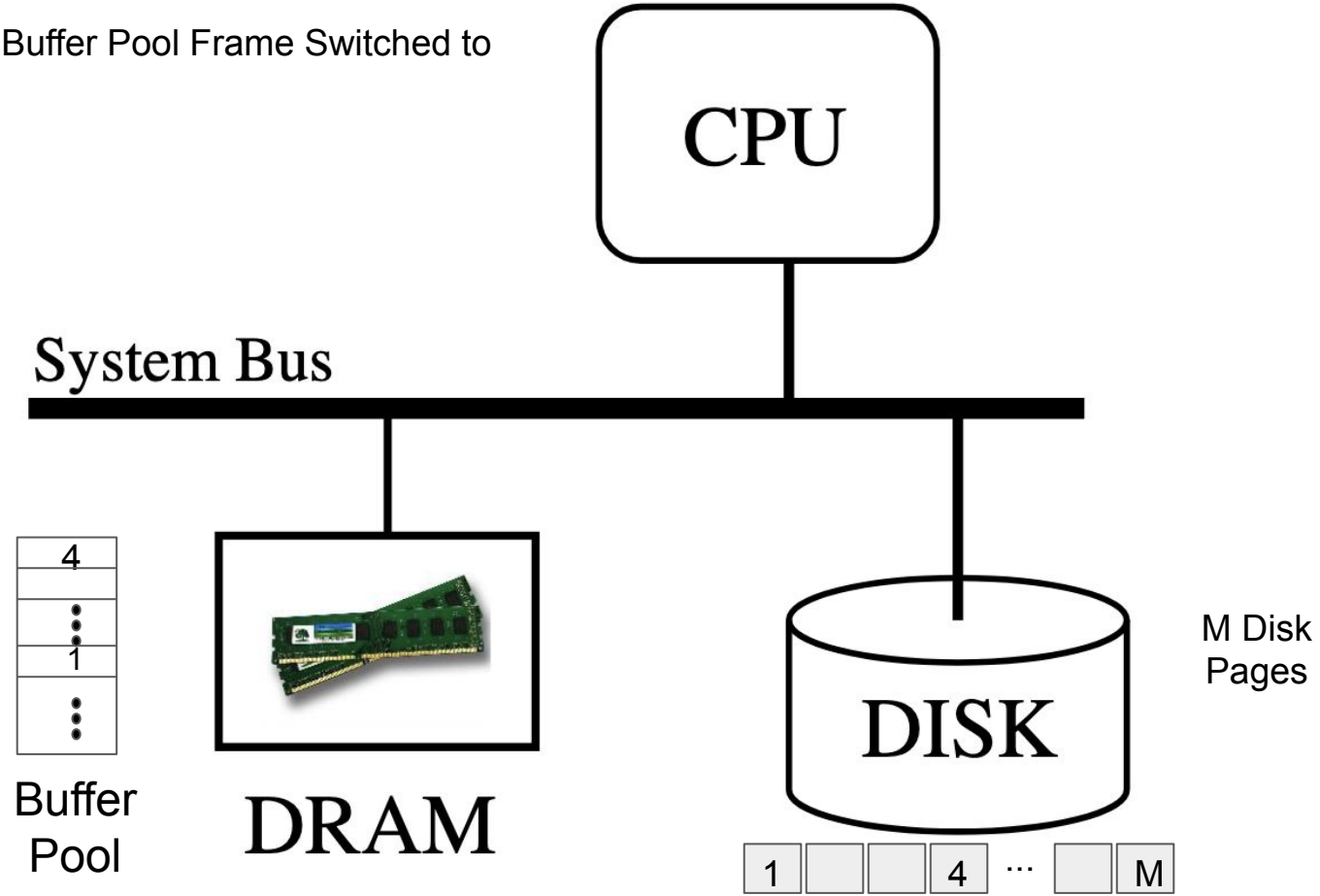


Disk Input/Output (I/O): Flush Page 4 to Disk



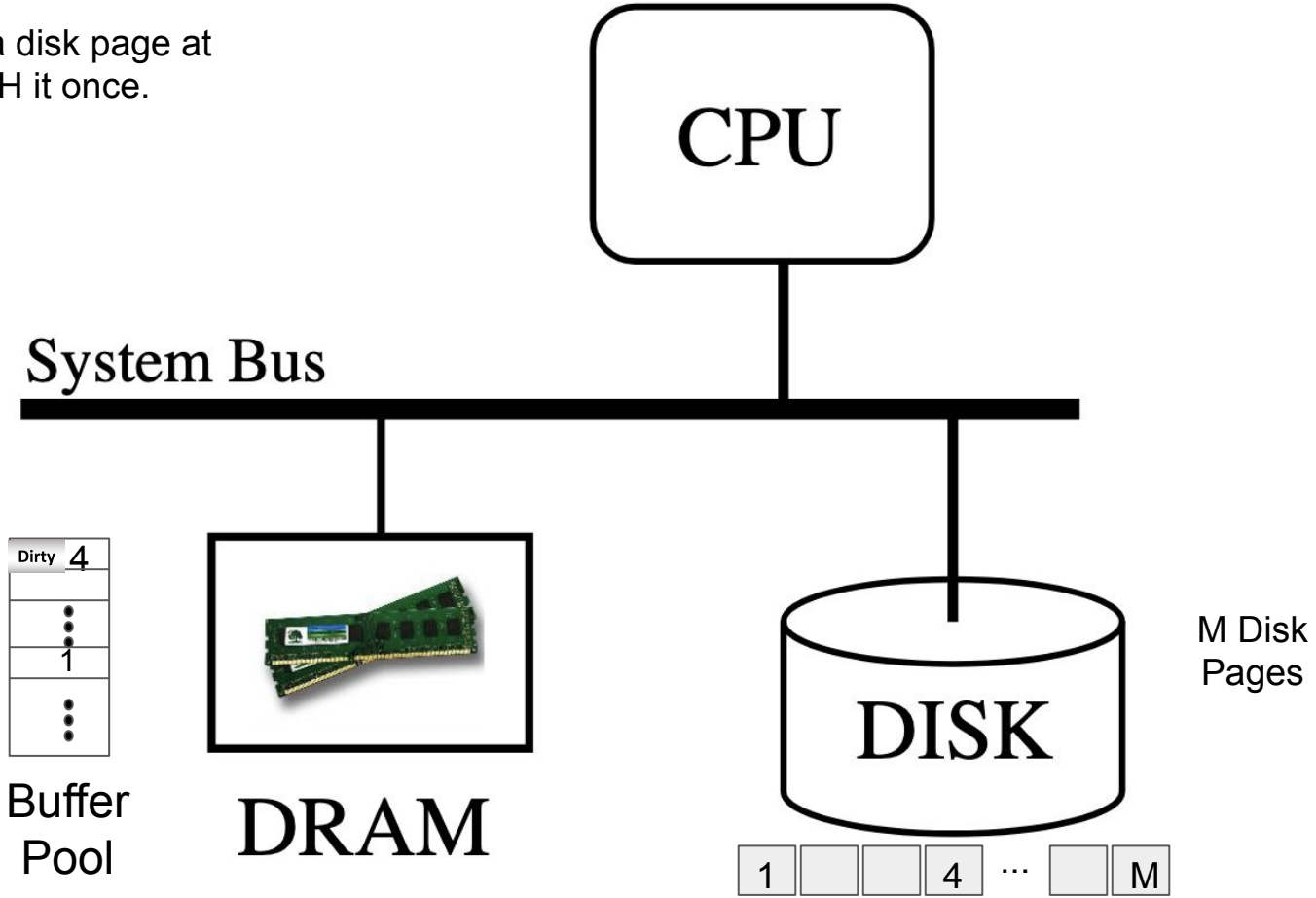
Disk Input/Output (I/O): Dirty becomes Clean

Flush complete: Dirty Buffer Pool Frame Switched to Clean.



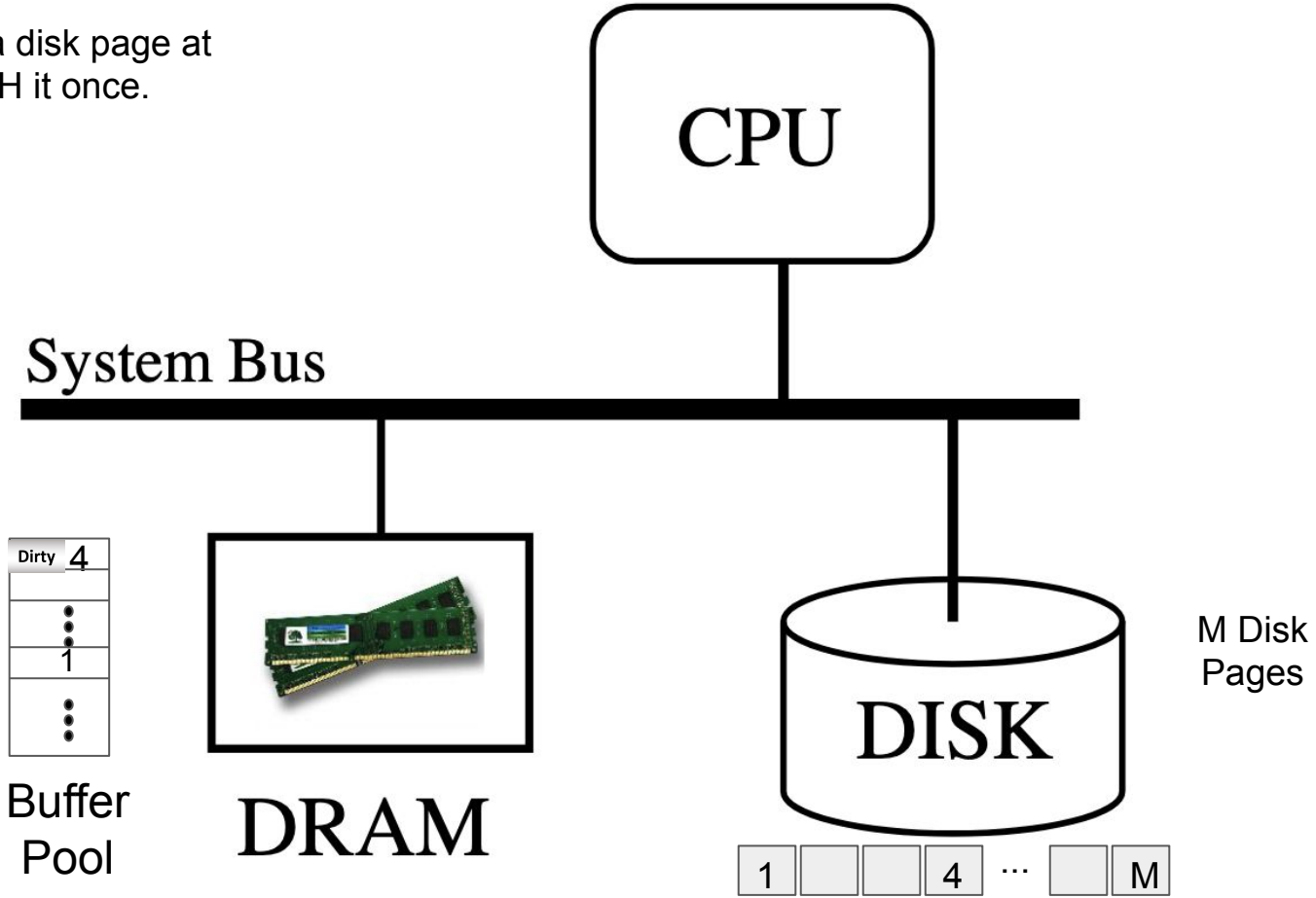
Disk Input/Output (I/O)

The DBMS may write a disk page at different times & FLUSH it once.

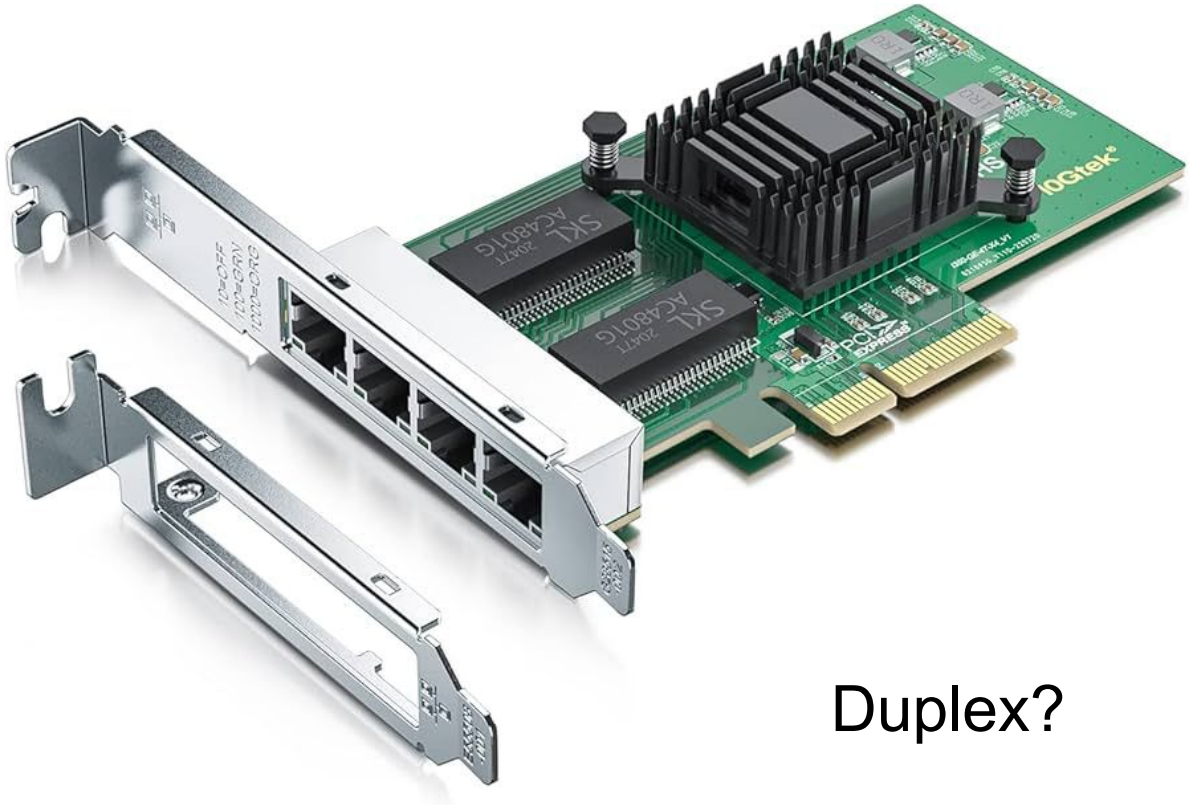


Disk Input/Output (I/O)

The DBMS may write a disk page at different times & FLUSH it once.



Network Interface Card, NIC





PCIe Gigabit Ethernet Server Adapter with Broadcom NetXtreme BCM5751 10/100/1000Mbps Gigabit Desktop PCI-E Network Card NIC

Visit the ULANSeN Store

4.4 ★★★★½ 122 ratings | Search this page

Amazon's Choice

in Internal Computer Networking Cards by ULANSeN

100+ bought in past month

$19.99

FREE Returns

Save up to 4% with business pricing. Sign up for a free Amazon Business account

Get $60 off instantly: Pay $0.00 $19.99 upon approval for the Amazon Store Card. No annual fee.

Brand	ULANSeN
Hardware Interface	PCI Express x1, Ethernet
Color	5751
Compatible Devices	Desktop
Product Dimensions	5"L x 0.8"W x 5"H

Captured Sept 5, 2024



Shop All Aomoproing Products | Visit ZY Electron MARKETPLACE Store

FOLLOW

1G 10/100/1000Mbps Gigabit Ethernet Converged Network Adapter (NIC) with Broadcom BCM5751 Chip | Ethernet PCI Express Network Card | Single RJ45 Port | PCI Express X1 | Compare to Broadcom BCM5751-T1

Be the first to review this product...

Ask Or Answer A Question

$44.00

Free 30-day Returns

Pay with Newegg Store Credit Card. Learn how AND Save 4% off* purchase. See details

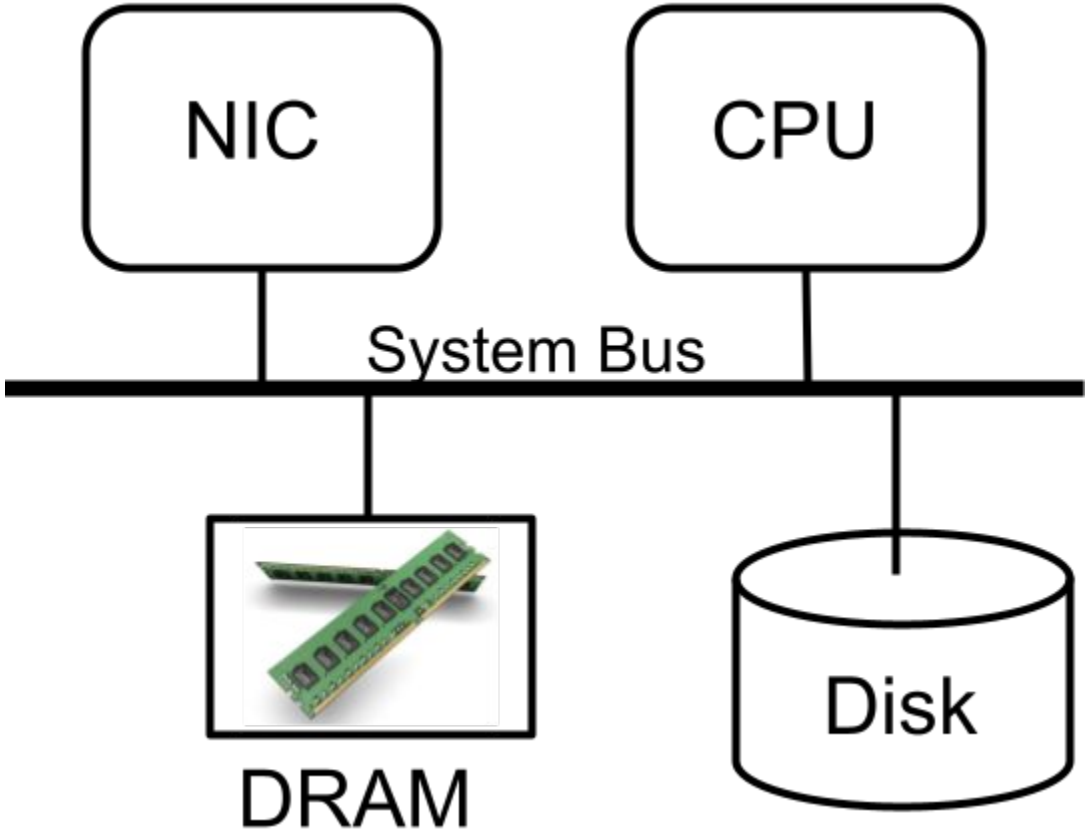
Pay in 4 interest-free payments of $11.00. Learn more

− 1 + Add to cart

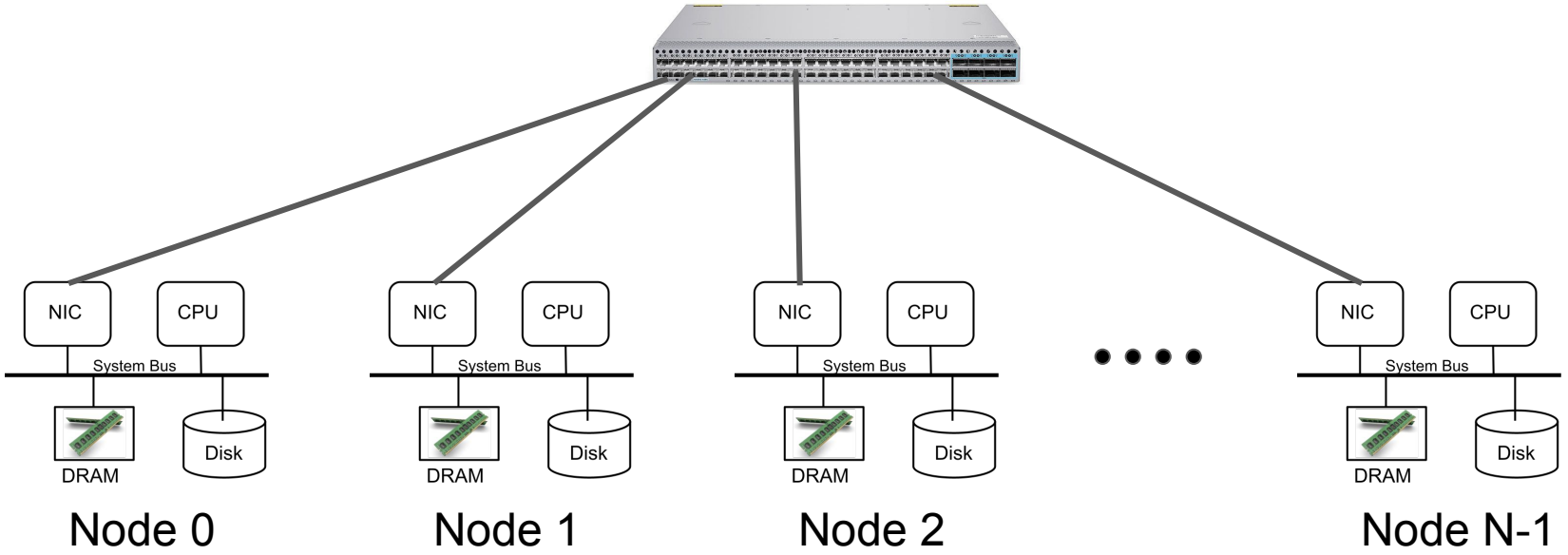
Captured Aug 31, 2026

Duplex?

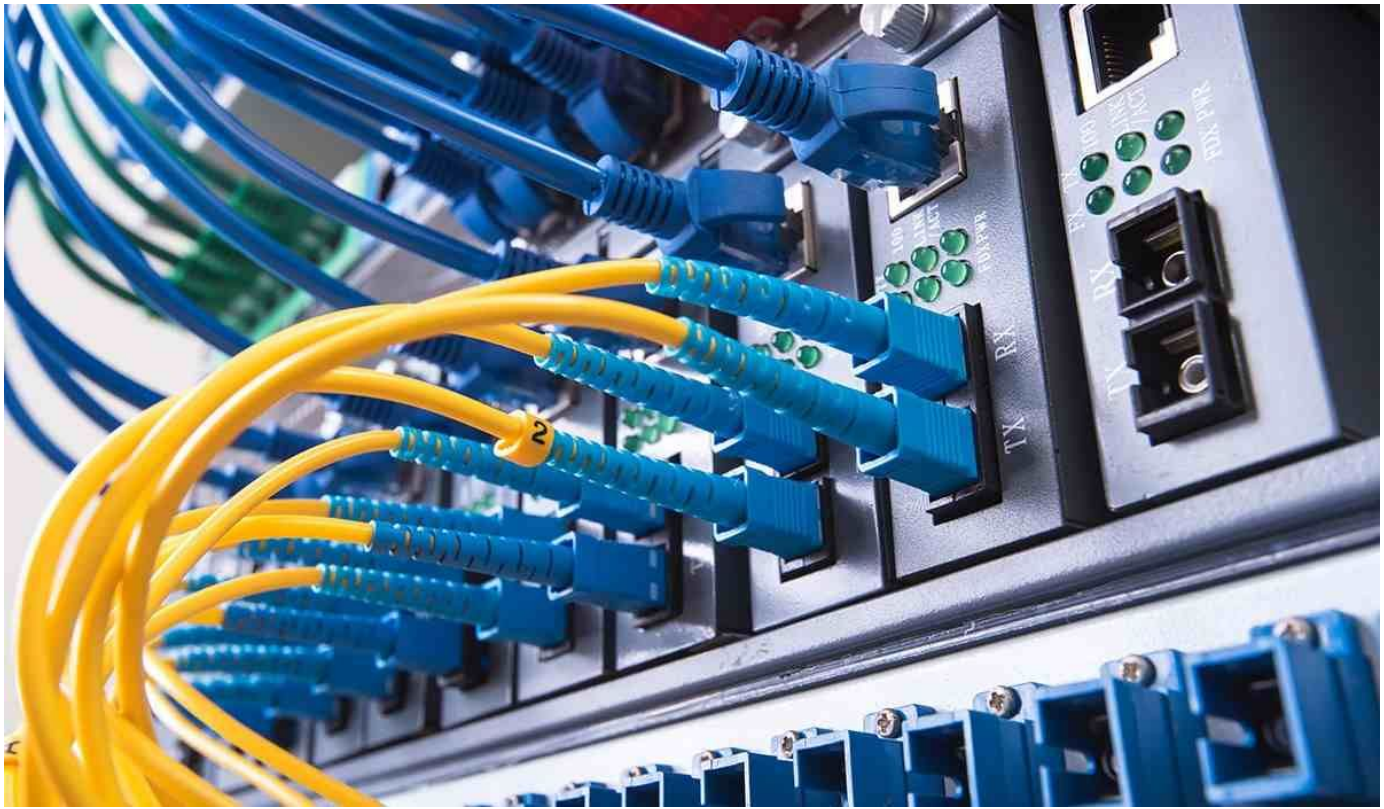
A Network Attached Node



A Shared-Nothing Architecture



In Theory

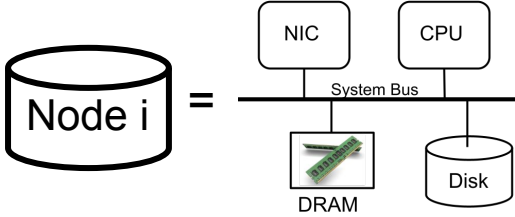


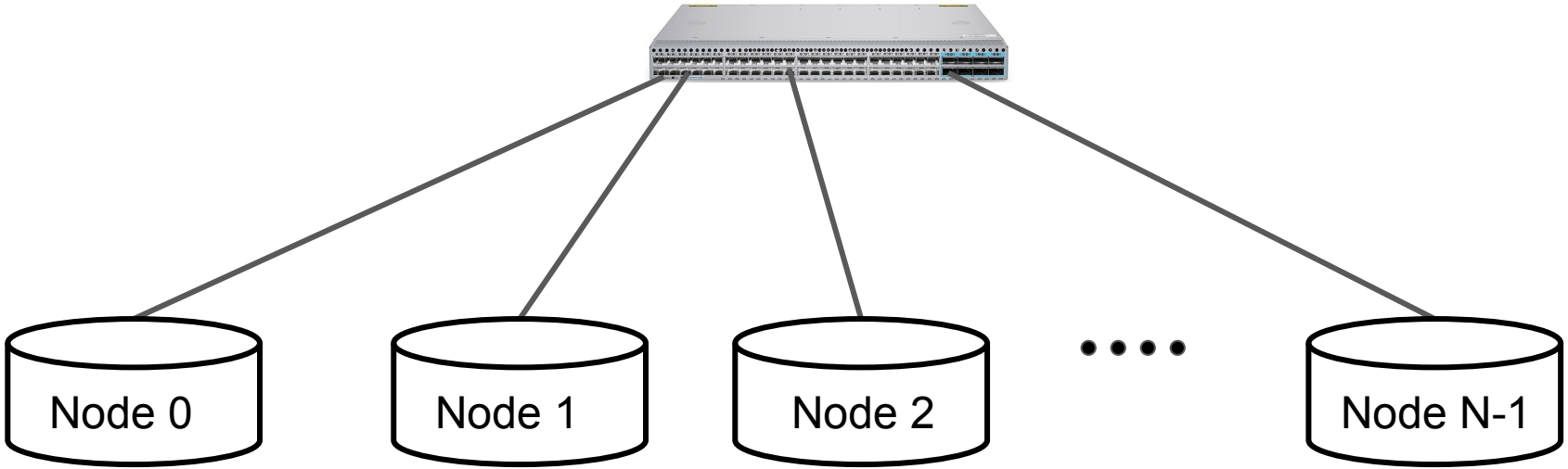
In Practice





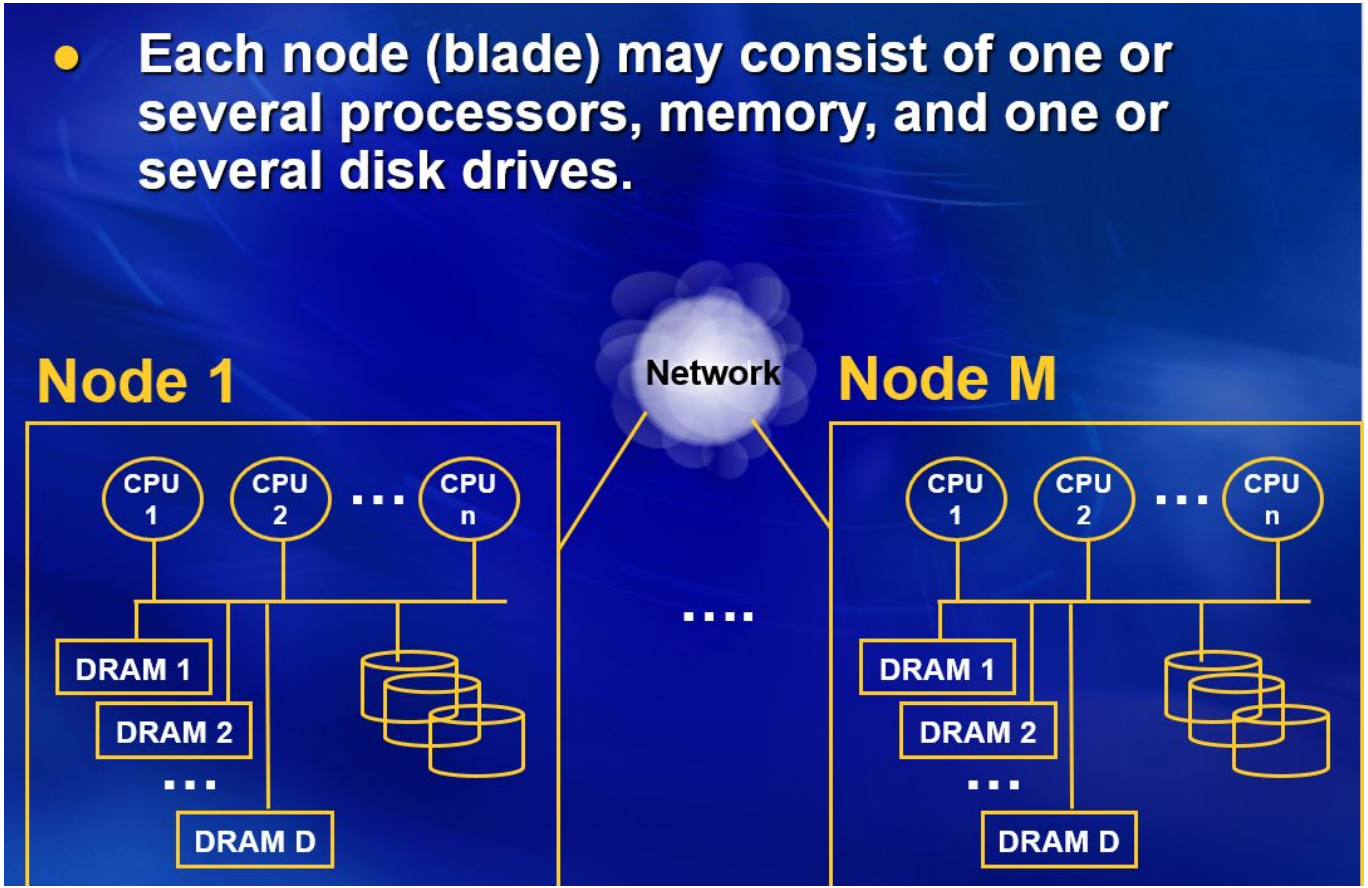
A Shared-Nothing Architecture





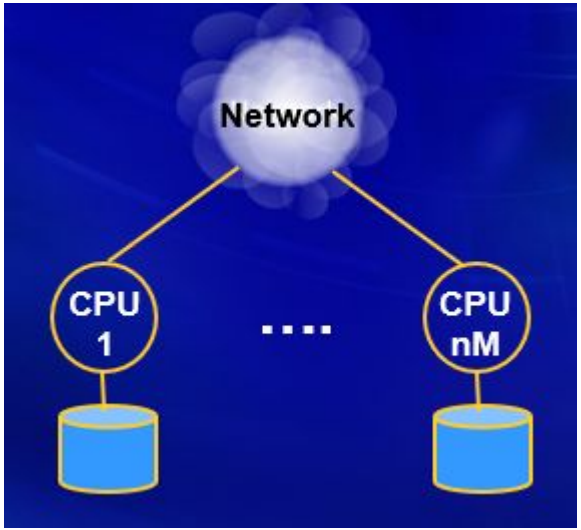
Virtual Nodes: Motivation

- Each node (blade) may consist of one or several processors, memory, and one or several disk drives.



Virtual Nodes

Partition resources to construct logical nodes. With an 8 core processor, construct eight logical nodes each with one processor, a fraction of memory, and one disk drive.



Why use multiple nodes?

Speedup: Perform a complex query/update faster. Ideally, it should become more than N times faster with N nodes. Example update/query:

1. Give all employees a 10% raise.
2. What is the average salary of employees with 2 or more children working in different departments.

Scaleup: Maintain the same speed for a complex query/update as the database grows by increasing the number of nodes. Example:

- Consider the above query. If the database grows 10x, maintain the same speed by increasing the number of nodes 10x.

How?

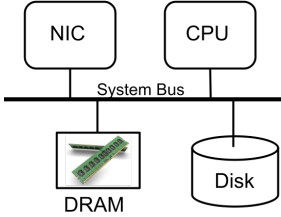
Design philosophy:

- Divide and conquer by partitioning the data.
- Use decentralized algorithms.
- Minimize coordination.

Challenges:

- Nodes fail.
- Latency associated with network communication.
- Design and implementation of decentralized algorithms is non-trivial.

Programs, Processes & Threads



Program: A passive, static file saved on persistent storage containing instructions for execution by the CPU.

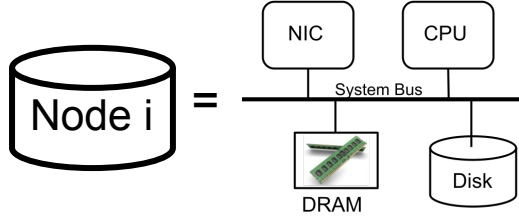
Process: An active, dynamic execution created when the Operating System loads a program into memory and runs it. One may execute a program multiple times to create multiple distinct processes.

Thread: A unit of execution inside a process. Threads within the same process may share memory (data structures) and OS resources such as files and sockets.

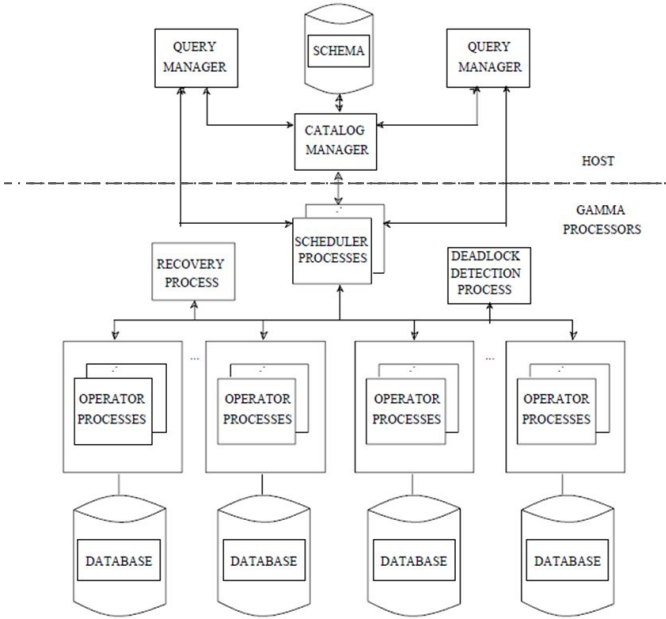
For rapid prototyping, use frameworks for building concurrent, distributed, resilient, and elastic applications. Two examples:

- Vert.x is an event-driven, non-blocking architecture. See https://start.vertx.io/ for details.
- Apache Pekko based on the Actor Model. See https://pekko.apache.org/docs/pekko/1.3/typed/guide/introduction.html for details.

Gamma Architecture



- Each node stores its fragment on its local disk drive
- Each node may build a B+-tree (clustered/non-clustered) and hash index on its fragment of a table.
- Each node has its own concurrency control and crash recovery protocol.

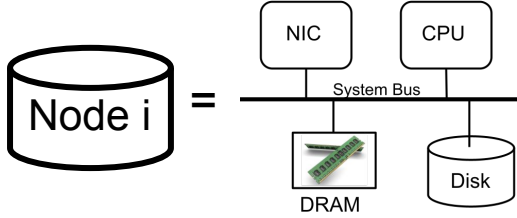



```
SELECT    *
FROM      Emp
WHERE     salary=60K
```



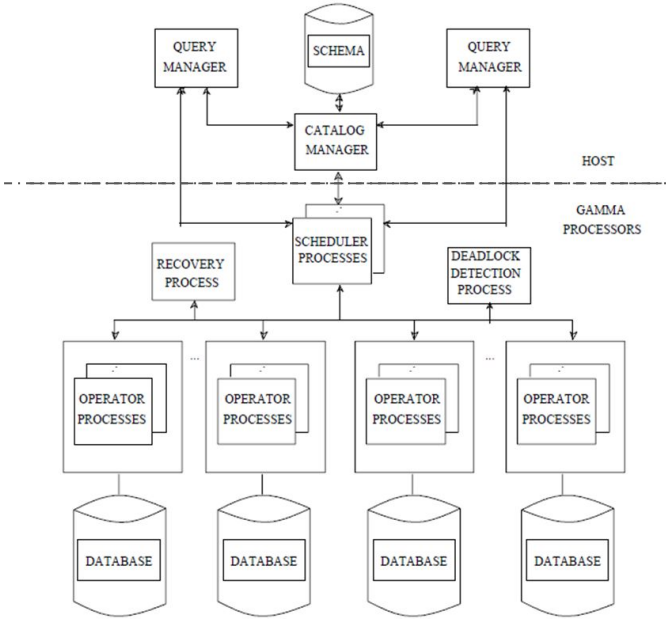


Gamma Architecture



- Each node stores its fragment on its local disk drive
- Each node may sh index on its fragment of a
- Each node has its own concurrency control and crash recovery protocol.

SQL to hide parallelism: User is not aware of parallel processing.

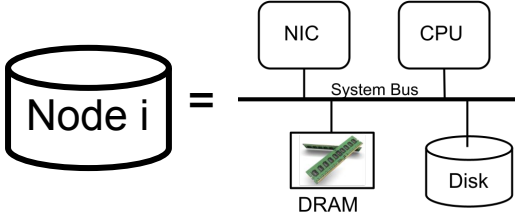


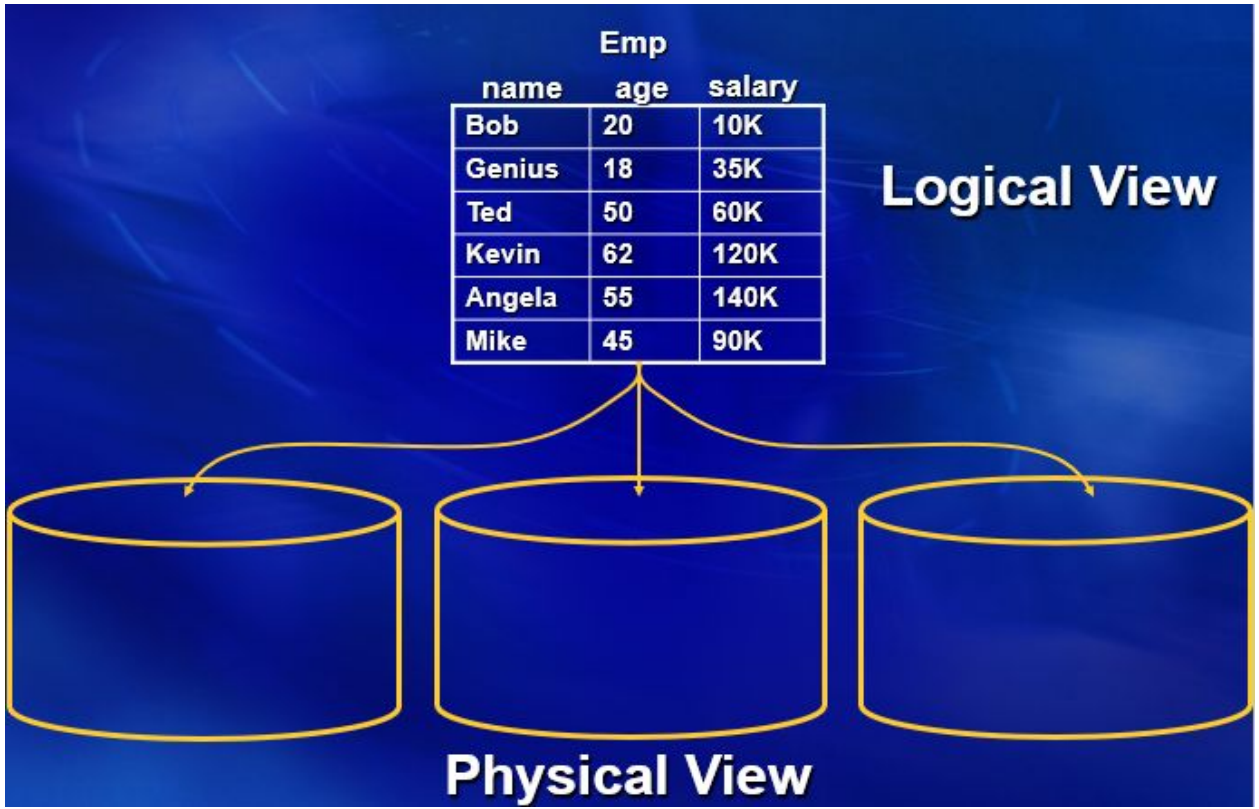

```
SELECT    *
FROM      Emp
WHERE     salary=60K
```



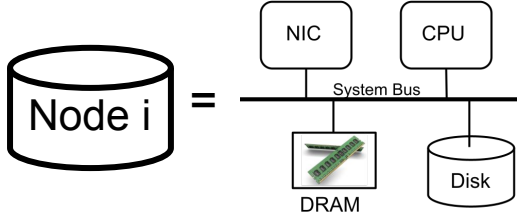


Horizontal Declustering/Partitioning/Sharding





Horizontal Declustering: How?



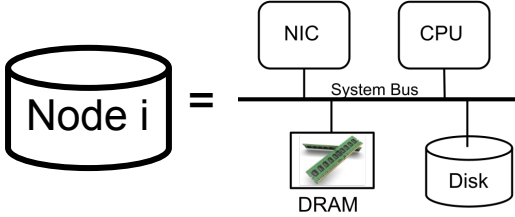
No partitioning attribute: Random and Round-robin.

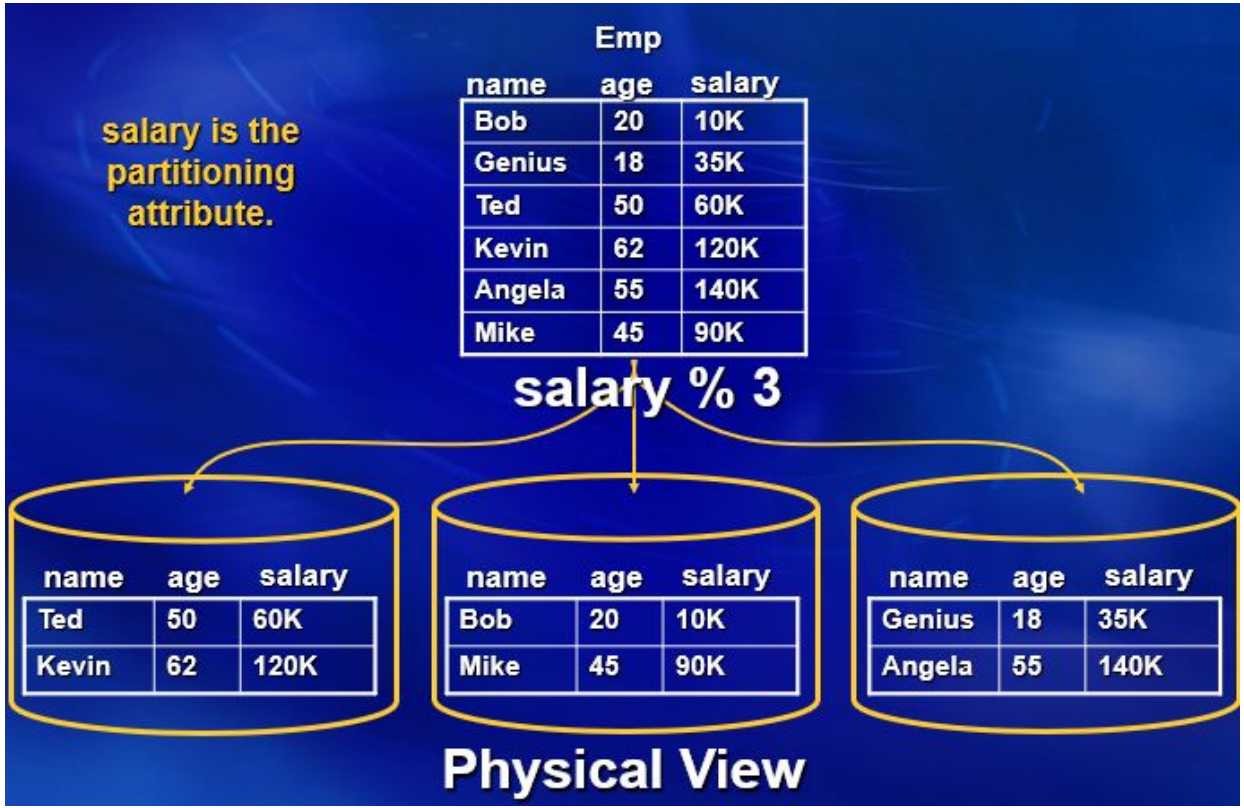
Single attribute declustering strategy:

1. Hash
2. Range

Note: The administrator must select an attribute as the partitioning attribute.

Hash Declustering/Partitioning/Sharding





What is the MOD (%) Function?

F(x) = x MOD N = x % N

- It is the remainder of x divided by N. Assume N=10:
  - F(12) = 12 % N = 2
  - F(10) = 10 % N = 0
  - F(1) = 1 % N = 1
  - F(101) = 101 % N = 1
  - F(20,511) = 20,511 % N = 1

- x MOD N maps a value x to a number between 0 and N-1.
- It is simple and perhaps the most popular hash functions out there.

Hash Declustering

Selections with equality predicates referencing the partitioning attribute are directed to a single node:

- Retrieve Employees with a 60K salary

```
SELECT *
FROM   Emp
WHERE salary=60K
```

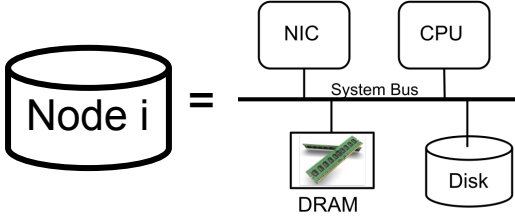
Equality predicates referencing a non-partitioning attribute and range predicates are directed to all nodes:

- Retrieve Employees who are 20 years old
- Fetch Employees with a salary lower than 50K

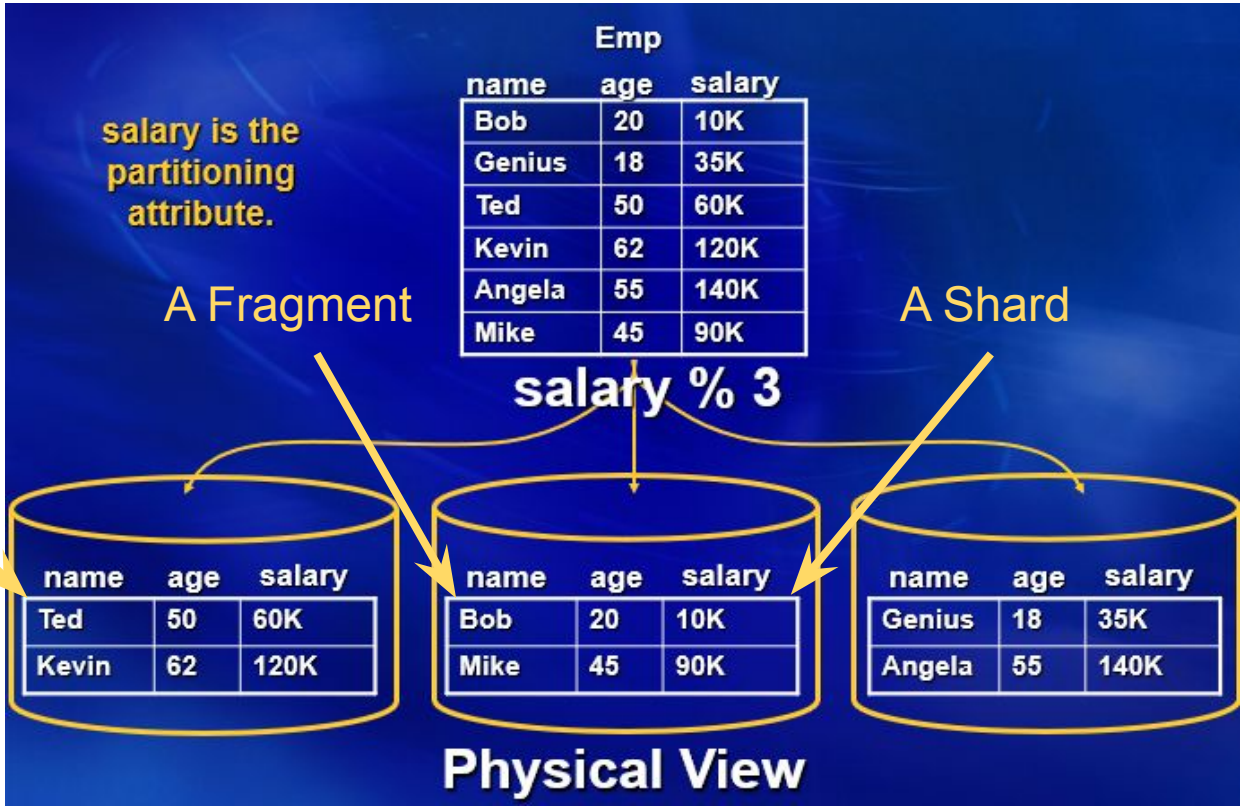
```
SELECT *
FROM   Emp
WHERE age=20
```

```
SELECT *
FROM   Emp
WHERE salary < 50K
```

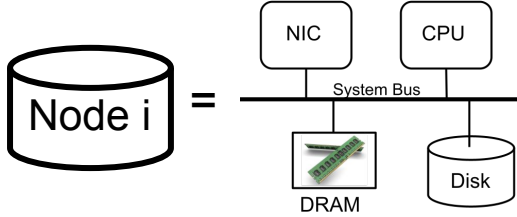
A Fragment/Shard



A Fragment



Range Declustering/Partitioning/Sharding



salary is the partitioning attribute.

Emp

name	age	salary
Bob	20	10K
Genius	18	35K
Ted	50	60K
Kevin	62	120K
Angela	55	140K
Mike	45	90K

0-50K

name	age	salary
Bob	20	10K
Genius	18	35K

51K-100K

name	age	salary
Ted	50	60K
Mike	45	90K

101K-∞

name	age	salary
Kevin	62	120K
Angela	55	140K

Physical View

Range Declustering

Equality and range predicates referencing the partitioning attribute are directed to a subset of nodes:

- Retrieve Employees with a 60K salary
- Fetch Employees with a salary lower than 50K

```
SELECT *
FROM   Emp
WHERE salary=60K
```

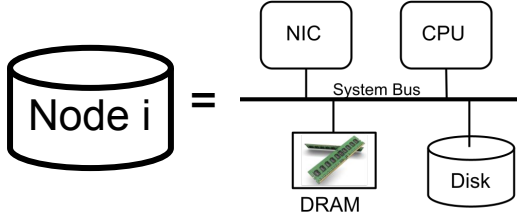
```
SELECT *
FROM   Emp
WHERE salary < 50K
```

Predicates referencing a non-partitioning attribute are directed to all nodes:

- Retrieve Employees who are 20 years old

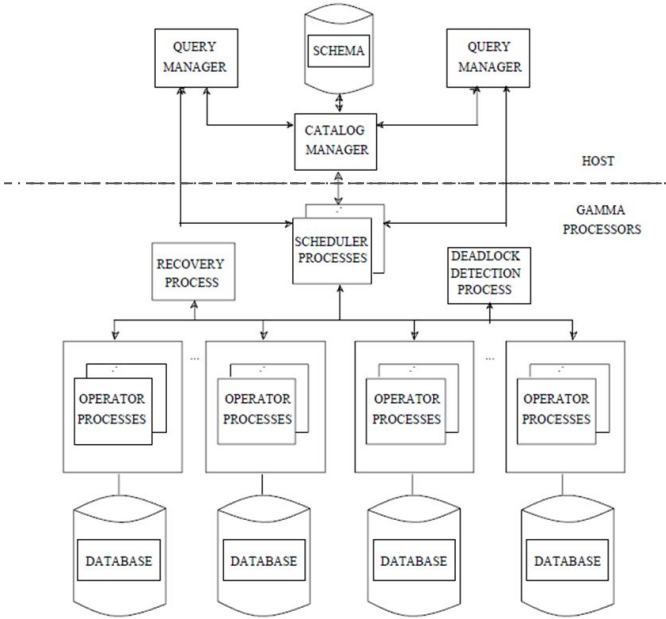
```
SELECT *
FROM   Emp
WHERE age=20
```

Gamma Architecture



- Each node stores its fragment on its local disk drive
- Each node may sh index on its fragment of a
- Each node has its own concurrency control and crash recovery protocol.

SQL to hide parallelism: User is not aware of parallel processing.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```



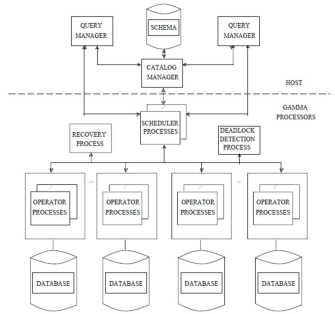


SQL

Shahram Ghandeharizadeh
CSCI 585

SQL Data Definition Language

A "What"-oriented language. The system is responsible for "How".



It allows the specification of not only a set of relations but also information about each relation, including:

- The schema for each relation.
- The domain of values associated with each attribute.
- Integrity constraints
- The set of indices to be maintained for each relations.
- Security and authorization information for each relation.
- The physical storage structure of each relation on disk.

Domain Types in SQL

- **char(n).** Fixed length character string, with user-specified length *n*.
- **varchar(n).** Variable length character strings, with user-specified maximum length *n*.
- **int.** Integer (a finite subset of the integers that is machine-dependent).
- **smallint.** Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d).** Fixed point number, with user-specified precision of *p* digits, with *n* digits to the right of decimal point.
- **real, double precision.** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least *n* digits.

SQL INTRODUCTION

- For the purposes of the following, assume the existence of these two relations:
  Emp (SS#, name, age, salary, dno)
  Dept (dno, dname, floor, mgrSS#)

- **Structured Query Language** (**SQL**) consists of four basic commands:
  Select, Insert, Update, and Delete.

- SQL provides commands to change the state of database: table creation/deletion, **insert**, **delete**, and **update**

CREATE TABLE COMMAND

- **Emp** (SS#, name, age, salary, dno)
  Dept (dno, dname, floor, mgrSS#)

- The allowed integrity constraints include:
  - Primary key (Aj1, Aj2, …, Ajm): states that attributes Aj1, Aj2, …,Ajm constitute the primary key for relation r: non-null and unique.
  - Check(P): states that every record in the relation must satisfy the specified predicate P, e.g., P = "age > 10"

- Create table Emp (SS# int, name char(15), age int, salary int, dno int, **primary key** (SS#), **check** (salary > 24000))
- Create table Dept (dno int, dname char(15), floor int, mgrss# int, **primary key** dno)

Basic Query Structure

- SQL is based on set and relational operations with certain modifications and enhancements
- A typical SQL query has the form:

select $A_1, A_2, ..., A_n$
from $r_1, r_2, ..., r_m$
where P

  - A_i represents an attribute
  - R_i represents a relation
  - P is a predicate.

- The result of an SQL query is a relation.

SELECT COMMAND

- The select command has the following syntax:
  select [**distinct**] target-list
  from tuple variable list
  where qualification
  [**order by** target list subset]
  [**having** set-qualification]
  the items in square brackets ([ ]) are electives.

- Qualification list can be a boolean combination (and, or, not) of either selection and join clauses.

- A ***selection clause*** is a comparison between an indexed tuple variable and a constant. A comparison operator might be {=, ≠, ≤, ≥, >, <}.

SELECT COMMAND (Cont…)

- A ***join clause*** is a comparison between two indexed tuple variable. Once again, a comparison operator might be {=, ≠, ≤, ≥, >, <}. In the following example, e.dno=d.dno is a join clause while d.dname=`Toy' is a selection clause:

  select e.name

  from Emp e, Dept d

  where e.dno = d.dno and d.dname = ‘Toy’

select e.name
from Emp e, Dept d
where e.dno = d.dno and d.dname = ‘Toy’

Result = ?

Emp

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1

Dept

dno	dname	floor	mgrssnum
1	shoe	1	323
2	toy	1	323
3	men	2	323
4	women	2	324

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1

dname = “Toy”

dno	dname	floor	mgrssnum
2	toy	1	323

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1

dname = “Toy”

dno	dname	floor	mgrssnum
2	toy	1	323

Emp.dno = Dept.dno and d.dname = “Toy”

ssnum	name	age	salary	dno	dname	floor	mgrss#
324	John	40	55000	2	Toy	1	323

SELECT COMMAND (Cont…)

- To eliminate duplicates from the result, use the keyword distinct in the target list. For example, the following query shows all distinct salaries above 50,000 with no duplicates:

  select distinct e.salary

  from Emp e

  where e.salary > 50000

SELECT COMMAND (Cont…)

Conceptualize the execution of a select query as one of:

1. evaluate qualification to locate qualifying tuples,
2. project out columns of these tuples using the target-list,
3. make the resulting tuples into a relation and eliminate duplicates if necessary.

select [**distinct**] target-list
from tuple variable list
where qualification

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of those employees younger than 18 years old.

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of those employees younger than 18 years old.

```
SELECT   e.name
FROM     Emp e
WHERE    e.age < 18
```

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of those employees working on the first floor.

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of those employees working on the first floor.

```
SELECT   e.name
FROM     Emp e, Dept d
WHERE    e.dno = d.dno and d.floor=1
```

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of the employee managing the Toy department.

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of the employee managing the Toy department.

```
SELECT   e.name
FROM     Emp e, Dept d
WHERE    d.dname = Toy and d.mgrSS=e.SS#
```

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Name of the employee who are older than their manager.

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

Quick Review

- Name of the employee who are older than their manager.

```
SELECT   e.name
FROM     Emp emp, Dept d, Emp mgr
WHERE    e.dno=d.dno and d.mgrSS#=mgr.SS# and e.age>mgr.age
```

Relational Algebra + Query Tree

Shahram Ghandeharizadeh
CSCI 585

RELATIONAL ALGEBRA INTRODUCTION

- Assume the following two relations:

  Emp (SS#, name, age, salary, dno)

  Dept (dno, dname, floor, mgrSS#)

- Relational algebra is a procedural query language, i.e., user must define both "how" and "what" to retrieve. It is used to implement SQL.

- Relational algebra consists of a set of operators that consume either one or two relations as input. An operator produces one relation as its output.

- Unary operators include: select, project, and rename

- Binary operators include: cartesian product, equality join, natural join, join, semi-join, division, union, and set difference.

SELECT OPERATOR

- **Select (б)**: selects tuples that satisfy a predicate; e.g., retrieve the employees whose salary is 30,000

  $б_{Salary=30,000}(Employee)$

- Conjunctive ($\wedge$) and disjunctive ($\vee$) selection predicates are allowed;
  e.g., retrieve employees whose salary is higher than 30,000 and are younger than 25 years old:

  $б_{Salary>30,000 \wedge\ age<25}(Employee)$

- Note that only selection predicates are allowed. A selection predicate is either (1) a comparison ($=, \neq, \leq, \geq, <, >$) between an attribute and a constant (e.g., salary = 30,000) or (2) a comparison between two different attributes of the same relation (e.g., salary = age × 100).
- Note: This operator is different than the SELECT command of SQL.

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $б_{Salary=30,000}$(Employee)

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $б_{Salary=30,000}$(Employee)

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

SQL?

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $б_{Age>22}$(Employee)

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $б_{Age>22}$(Employee)

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
4	Kathy	30	30000	5

SQL?

PROJECT OPERATOR

- **Project** (∏) retrieves a column. It is a unary operator that eliminate duplicates.

  e.g., name of employees:

  $\prod_{name}(Employee)$

  e.g., name of employees earning more than 30,000:

  $\prod_{name}(б_{Salary>30,000}(Employee))$

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{age}(Emp)$

Age
24
20
22
30
4

SQL?

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{name,age}(б_{Salary=4000}\ (Emp)\ )$

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{name,age}(б_{Salary=4000}\ (Emp))$

SS#	Name	Age	Salary	dno
5	Genius	4	4000	1

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{\mathbf{name,age}}(б_{Salary=4000}\ (Emp)\ )$

Name	Age
Genius	4

SQL?

RENAME OPERATOR

- Rename operator changes the name of its input table to its subscript,
  - ρ_{e2}(Emp)
  - Changes the name of Emp table to e2

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN

- Equality join connects tuples from two relations that match on certain attributes. The specified joining columns are kept in the resulting relation.
  - $\prod_{name}(б_{dname='toy'}(Emp \bowtie_{(dno)} Dept)))$
- Natural join connects tuples from two relations that match on the specified common attributes
  - $\prod_{name}(б_{dname='toy'}(Emp \bowtie_{(dno)} Dept)))$
- How is an equality join between Emp and Dept using dno different than a natural join between Emp and Dept using dno?
  - Equality join: SS#, name, age, salary, Emp.dno, Dept.dno, …
  - Natural join: SS#, name, age, salary, dno, dname, …
- Join is similar to equality join using different comparison operators
  - $A \bowtie S \qquad op = \{=, \neq, \leq, \geq, <, >\}$
    att op att

EXAMPLE JOIN

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	1
3	Bob	22	27000	1
4	Kathy	30	30000	2
5	Genius	4	4000	1

EMP

dno	dname	floor	mgrss#
1	Toy	1	5
2	Shoe	2	1

Dept

- Equality Join, (Emp $\bowtie_{(dno)}$ Dept)))

SS#	Name	Age	Salary	EMP.dno	Dept.dno	dname	floor	mgrss#
1	Joe	24	20000	2	2	Shoe	2	1
2	Mary	20	25000	1	1	Toy	1	5
3	Bob	22	27000	1	1	Toy	1	5
4	Kathy	30	30000	2	2	Shoe	2	1
5	Genius	4	4000	1	1	Toy	1	5

EXAMPLE JOIN

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	1
3	Bob	22	27000	1
4	Kathy	30	30000	2
5	Genius	4	4000	1

EMP

dno	dname	floor	mgrss#
1	Toy	1	5
2	Shoe	2	1

Dept

- Natural Join, (Emp $\bowtie_{(dno)}$ Dept)))

SS#	Name	Age	Salary	dno	dname	floor	mgrss#
1	Joe	24	20000	2	Shoe	2	1
2	Mary	20	25000	1	Toy	1	5
3	Bob	22	27000	1	Toy	1	5
4	Kathy	30	30000	2	Shoe	2	1
5	Genius	4	4000	1	Toy	1	5

EXAMPLE JOIN

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	1
3	Bob	22	27000	1
4	Kathy	30	30000	2
5	Genius	4	4000	1

EMP

Salary > 5 * salary

dno	dname	floor	mgrss#
1	Toy	1	5
2	Shoe	2	1

Dept

- Join, (Emp $\bowtie$ ρ_x(Emp))))

  Salary > 5 * x.Salary

SS#	Name	Age	Salary	dno	x.SS#	x.Name	x.Age	x.Salary	x.dno
2	Mary	20	25000	1	2	Genius	4	4000	1
3	Bob	22	27000	1	3	Genius	4	4000	1
4	Kathy	30	30000	2	4	Genius	4	4000	1

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont…)

- Example, retrieve Joe's salary:

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont…)

- Example, retrieve Joe's salary:
  - $\Pi_{Salary}(б_{name='Joe'}(Emp))$

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont…)

- Example, retrieve the name of Joe's manager:

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont…)

- Example, retrieve the name of employees who earn more than Joe:

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont…)

- Example: retrieve the name of employees who earn more than Joe:
  - $\prod_{name}(Emp \bowtie_{(sal>x.sal)} б_{name='Joe'}(\rho_{x}(Emp)))$

SQL?

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EXAMPLE

- Assume a database with the following three relations:
  Sailors (sid, sname, rating)
  Boats (bid, bname, color)
  Reserve (sid, bid, date)

- Query 1: Find the boat id (bid) of red colored boats.

EXAMPLE

- Assume a database with the following three relations:
  Sailors (sid, sname, rating)
  Boats (bid, bname, color)
  Reserve (sid, bid, date)

- Query 1: Find the name of sailors who have reserved a red boat.

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

Quick Review

- Employees working for the Toy department

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

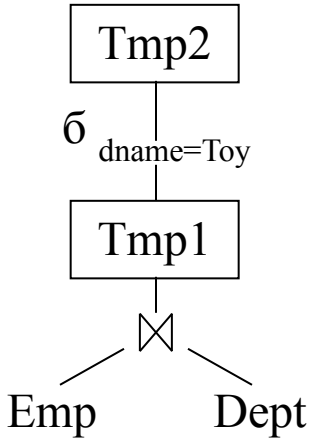
- Employees working for the Toy department:

$$TMP2 \leftarrow \sigma_{dname=Toy}(Emp \bowtie Dept)$$

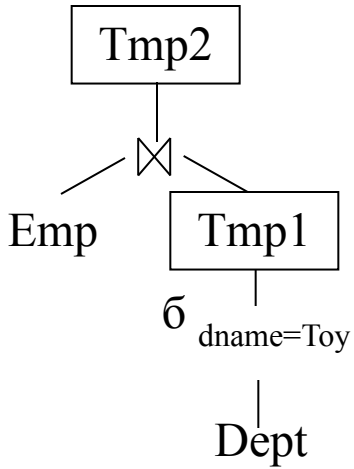
$$TMP2 \leftarrow Emp \bowtie \sigma_{dname=Toy}(Dept)$$

SQL?

- TMP2 ← $б_{dname=Toy}(Emp \bowtie Dept)$



- TMP2 ← $Emp \bowtie б_{dname=Toy}(Dept)$

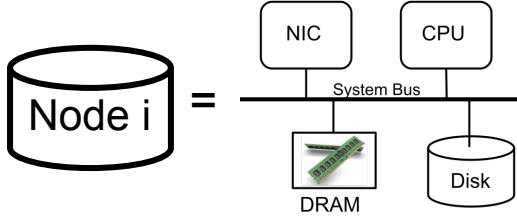


Gamma

by DeWitt et. al.

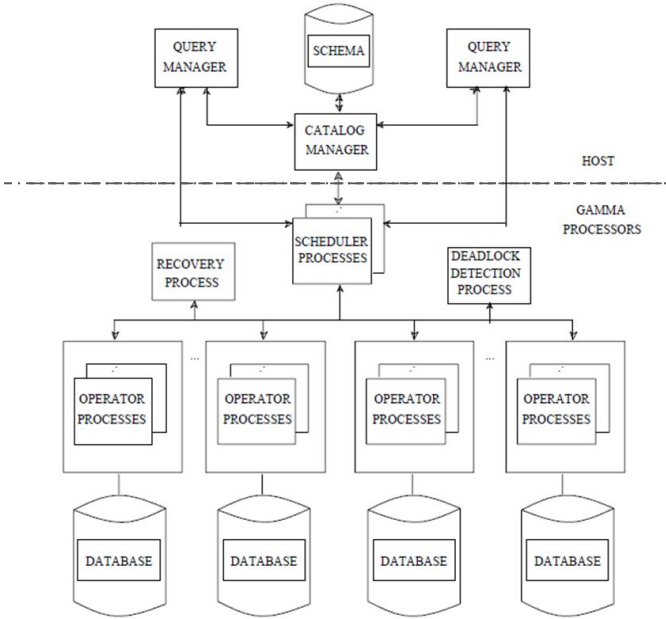
Shahram Ghandeharizadeh
CSCI 585

Gamma Architecture



- Each node stores its fragment on its local disk drive
- Each node may sh index on its fragment of a
- Each node has its own concurrency control and crash recovery protocol.

SQL to hide parallelism: User is not aware of parallel processing.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```

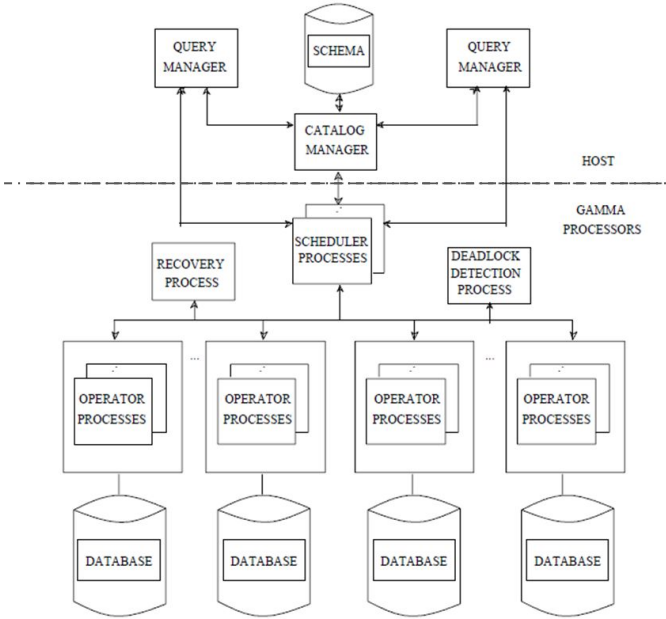




Architecture

Catalog Manager

The function of the Catalog Manager is to act as a central repository of all conceptual and internal schema information for each database. The schema information is loaded into memory when a database is first opened. Since multiple users may have the same database open at once and since each user may reside on a machine other than the one on which the Catalog Manager is executing, the Catalog Manager is responsible for insuring consistency among the copies cached by each user.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```





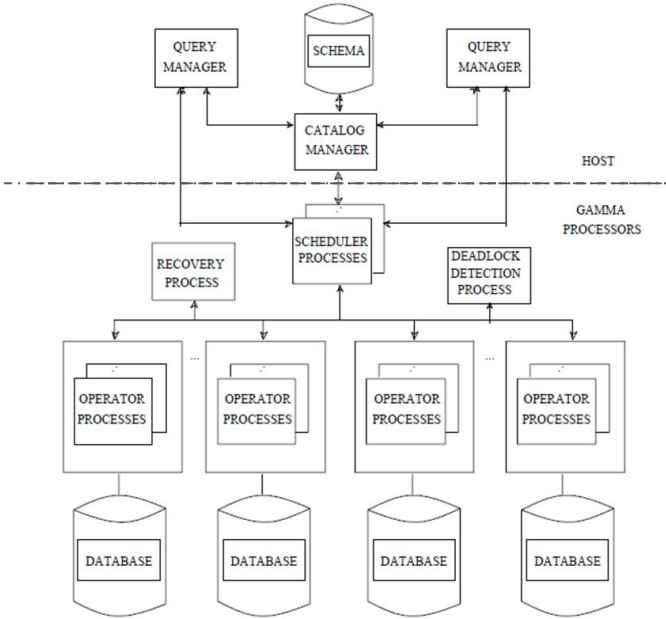
Architecture

- Each node stores its fragment on its local disk drive

Query Manager

One query manager process is associated with each active Gamma user. The query manager is responsible for caching schema information locally, providing an interface for ad-hoc queries using gdl (our variant of Quel [STON76]), query parsing, optimization, and compilation.

- Each node has its own concurrency control and crash recovery protocol.




```
SELECT   *
FROM     Emp
WHERE    salary=60K
```





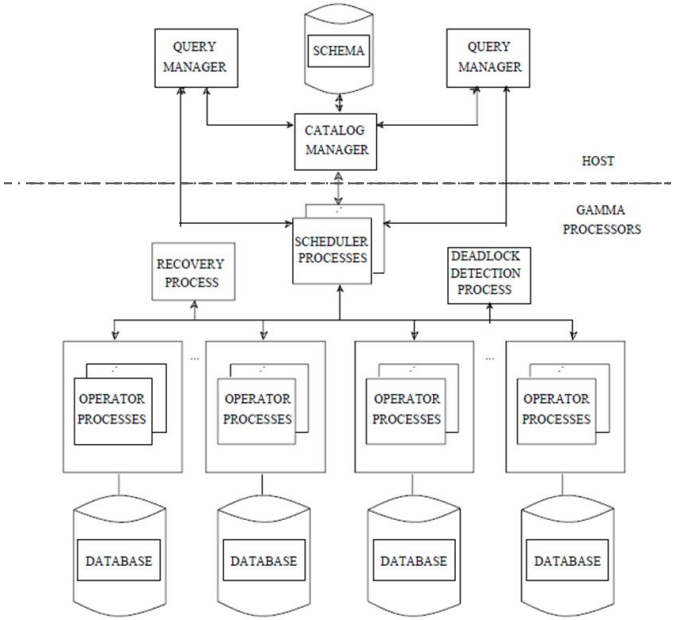
Architecture

- Each node stores its fragment on its local disk drive

Query Manager

One query manager process is associated with each active Gamma user. The query manager is responsible for caching schema information locally, providing an interface for ad-hoc queries using gdl (our variant of Quel [STON76]), query parsing, optimization, and compilation.

- Each node has its own concurrency control and crash recovery protocol.

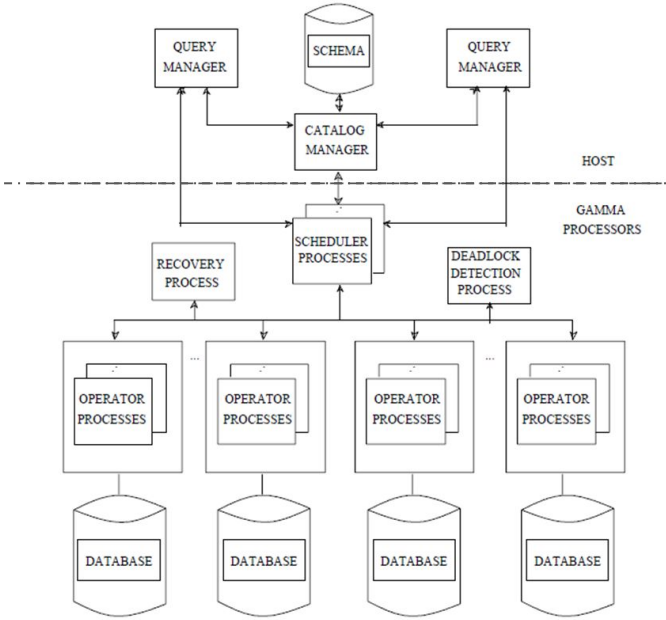


in the system. In the case of a single site query, the query is sent directly by the QM to the appropriate processor for execution. In the case of a multiple site query, the optimizer establishes a connection to an idle scheduler process through a centralized dispatcher process. The dispatcher process, by controlling the number of active schedulers, implements a simple load control mechanism. Once it has established a connection with a scheduler process, the QM sends the compiled query to the scheduler process and waits for the query to complete execution. The scheduler process, in turn, activates operator processes at each query processor selected to execute the operator. Finally, the QM reads the results of the query and returns them through the ad-hoc query interface to the user or through the embedded query interface to the program from which the query was initiated.

Architecture

Scheduler Processes

While executing, each multisite query is controlled by a scheduler process. This process is responsible for activating the Operator Processes used to execute the nodes of a compiled query tree. Scheduler processes can be run on any processor, insuring that no processor becomes a bottleneck. In practice, however, scheduler processes consume almost no resources and it is possible to run a large number of them on a single processor. A centralized dispatching process is used to assign scheduler processes to queries. Those queries that the optimizer can detect to be single-site queries are sent directly to the appropriate node for execution, by-passing the scheduling process.



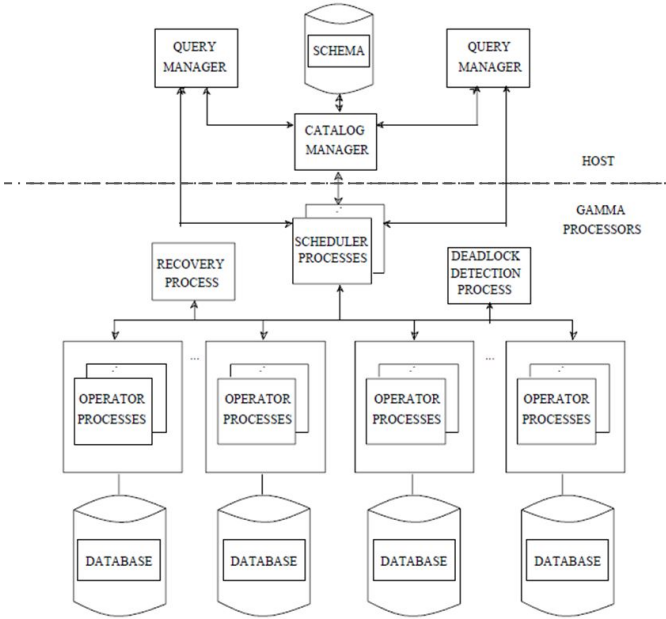

```
SELECT   *
FROM     Emp
WHERE    salary=60K
```





Operator Process

For each operator in a query tree, at least one Operator Process is employed at each processor participating in the execution of the operator. These operators are primed at system initialization time in order to avoid the overhead of starting processes at query execution time (additional processes can be forked as needed). The structure of an operator process and the mapping of relational operators to operator processes is discussed in more detail below. When a scheduler wishes to start a new operator on a node, it sends a request to a special communications port known as the "new task" port. When a request is received on this port, an idle operator process is assigned to the request and the communications port of this operator process is returned to the requesting scheduler process.




```
SELECT    *
FROM      Emp
WHERE     salary=60K
```





Software Architecture

- Processes executing on one node share memory - identical to today's threads.
- At initialization time, a node starts a fixed number of threads (processes).
- All threads listen on a well defined socket, waiting for the Scheduler to dispatch work to them.
- A message contains the identity that the operator should assume:
  - A switch statement enables a thread to become a select, a project, hash-join build, hash-join probe, etc.
  - The message specifies the role of the thread.

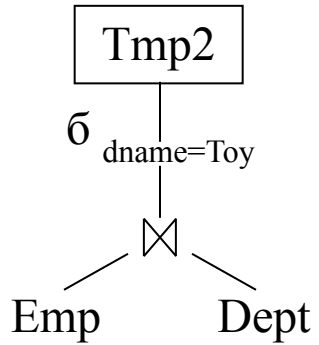
Operator Process

For each operator in a query tree, at least one Operator Process is employed at each processor participating in the execution of the operator. These operators are primed at system initialization time in order to avoid the overhead of starting processes at query execution time (additional processes can be forked as needed). The structure of an operator process and the mapping of relational operators to operator processes is discussed in more detail below. When a scheduler wishes to start a new operator on a node, it sends a request to a special communications port known as the "new task" port. When a request is received on this port, an idle operator process is assigned to the request and the communications port of this operator process is returned to the requesting scheduler process.

Pipelined Query Execution

1. Initiate a process for Store into TMP 2 operator.
2. Initiate a process for the select operator.
3. Initiate a Join process.
4. Read the Emp records and forward to the join process to build a hash table.
5. Read the Dept table and forward to the join process to probe the hash table. Forward each joining record to the Select process of Step 2.
6. Select operator prunes those records with dname=Toy and forwards them to the Store TMP2 process of Step 1.
7. TMP2 process writes qualifying records to disk.

- $TMP2 \leftarrow \sigma_{dname=Toy}(Emp \bowtie Dept)$



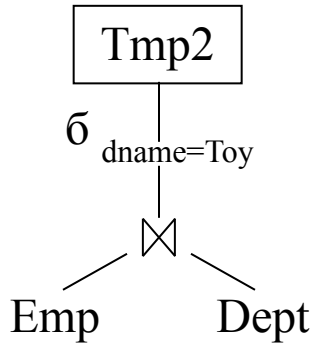

```
SELECT * into TMP2
FROM Emp e, Dept d
WHERE e.dno = d.dno and
d.dname = 'Toy'
```

Pipelined Query Execution

1. Initiate a process for Store into TMP 2 operator.
2. Initiate a process for the select operator.
3. Initiate a Join process.
4. Read the Emp records and forward to the join process to build a hash table.
5. Read the Dept table and forward to the join process to probe the hash table. Forward each joining record to the Select process of Step 2.
6. Select operator prunes those records with dname=Toy and forwards them to the Store TMP2 process of Step 1.
7. TMP2 process writes qualifying records to disk.

- $TMP2 \leftarrow \sigma_{dname=Toy}(Emp \bowtie Dept)$

Can we flip Emp and Dept?

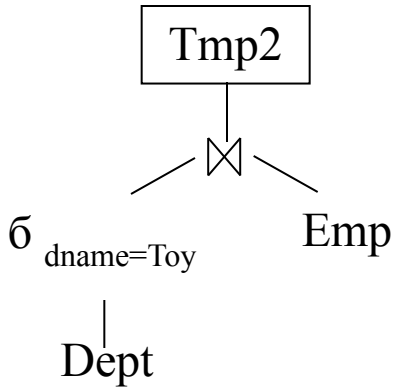



```
SELECT * into TMP2
FROM Emp e, Dept d
WHERE e.dno = d.dno and
d.dname = 'Toy'
```

Pipelined Query Execution

- TMP2 ← $\sigma_{dname=Toy}(Emp \bowtie Dept)$

1. Initiate a process for the Store into TMP 2 operator.
2. Initiate a process for the Join operator.
3. Initiate a process for the select operator that reads the Dept table and produces all the records with dname = Toy, forwarding them to the Join operator of Step 2.
4. The Join Build operator builds a hash table on dname of its received records.
5. Read the Emp record and forward to the Join operator. This operator probes the hash table for the qualifying record, forwarding them to the Store operator of Step 1.



Important Details

- The input and output of an operator is a stream of records.
- The select operator reads the records of a fragment and produce those satisfying a selection predicate as its output stream.
  - It is possible that no selection predicate is provided to the select operator. In this case, it becomes a simple ***scan*** that produces all records of a fragment.
- The join operator consists of two distinct steps.

Hash-Join Operator

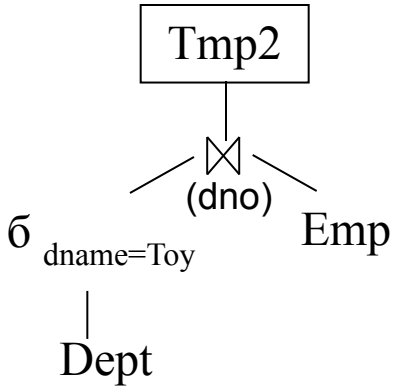
Consists of 2 distinct and sequential steps:

1. Build: Constructs a hash table on the join attribute.
2. Probe: Look up the hash table for a joining record.

In our example:

1. Build: Construct a hash table on the dno of Dept records with dname=Toy.
2. Probe: Look up the hash table with the Emp.dno of each Emp record.

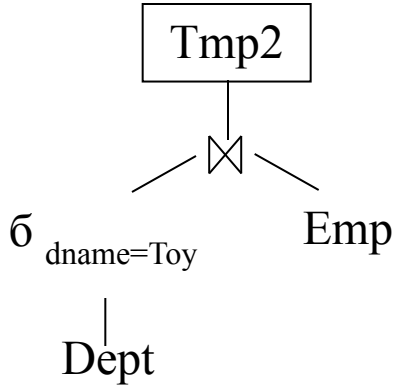
Sequential means the two steps follow one another. Probe begins after Build completes.

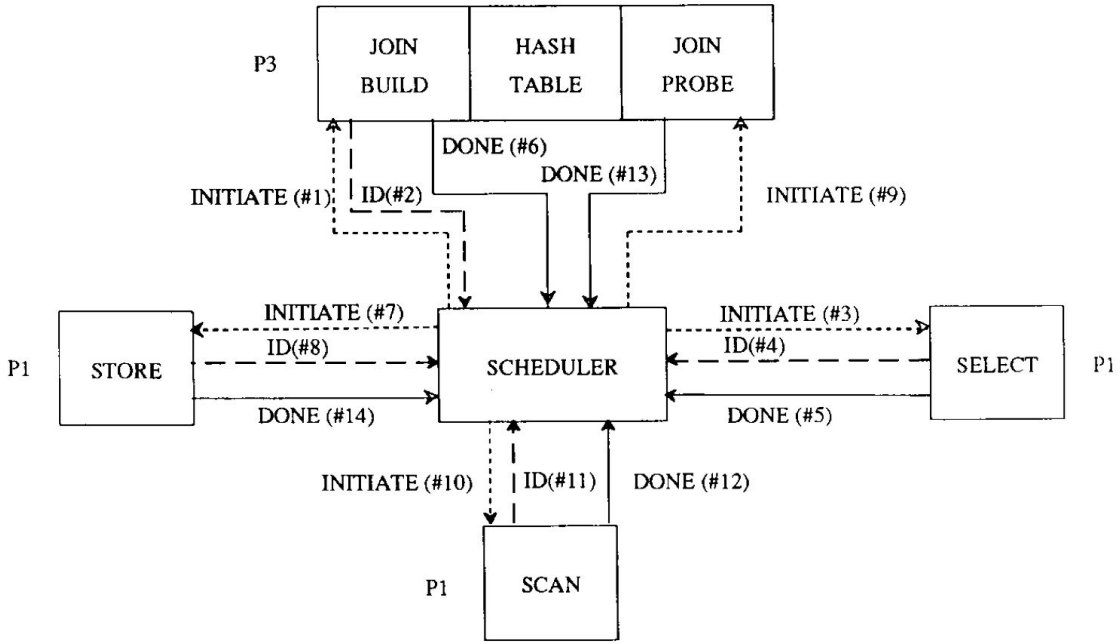


How does an operator know where to send its stream of records?

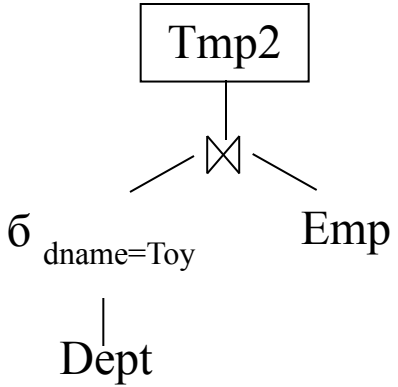
Scheduler Informs Them!

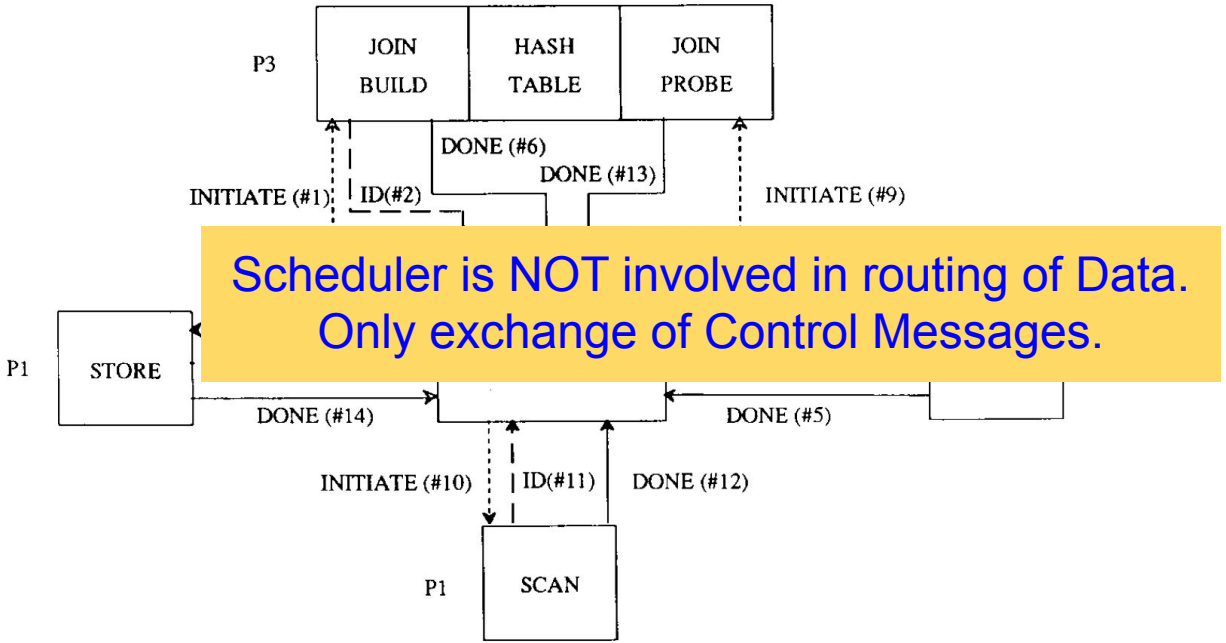
Scheduler controls and coordinates:





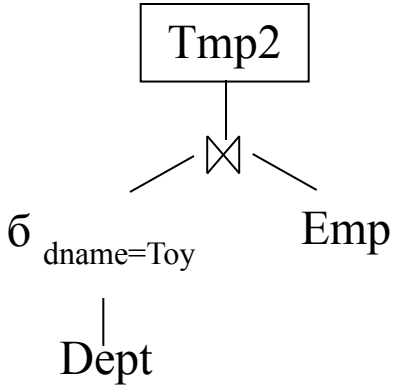
Scheduler Coordinates Flow of Data

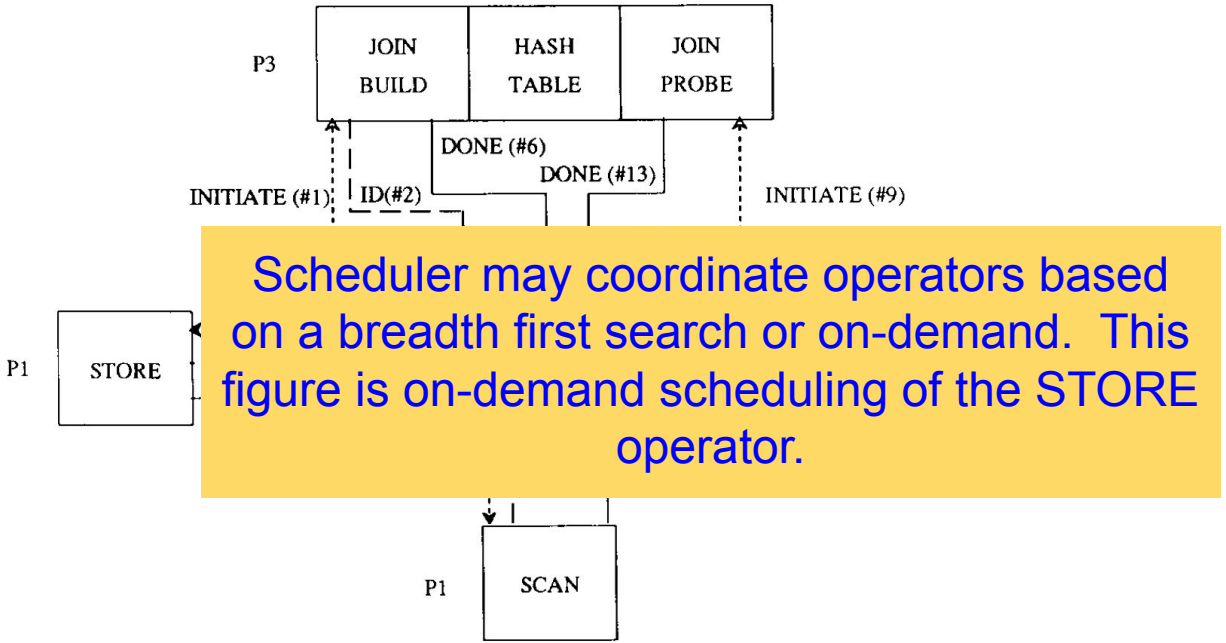




Scheduler is NOT involved in routing of Data.
Only exchange of Control Messages.

Scheduler Coordinates Flow of Data





Scheduler may coordinate operators based on a breadth first search or on-demand. This figure is on-demand scheduling of the STORE operator.

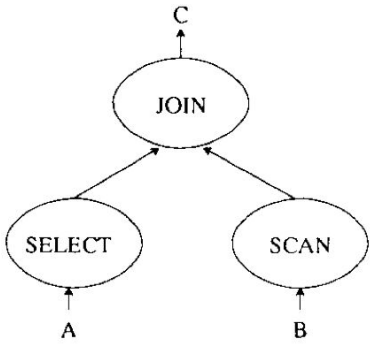
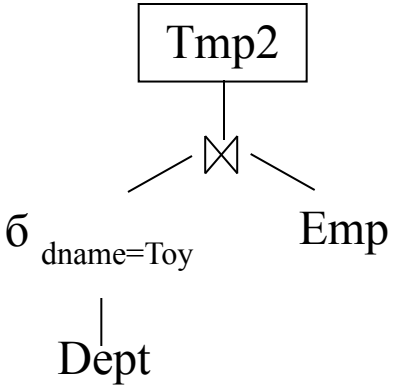
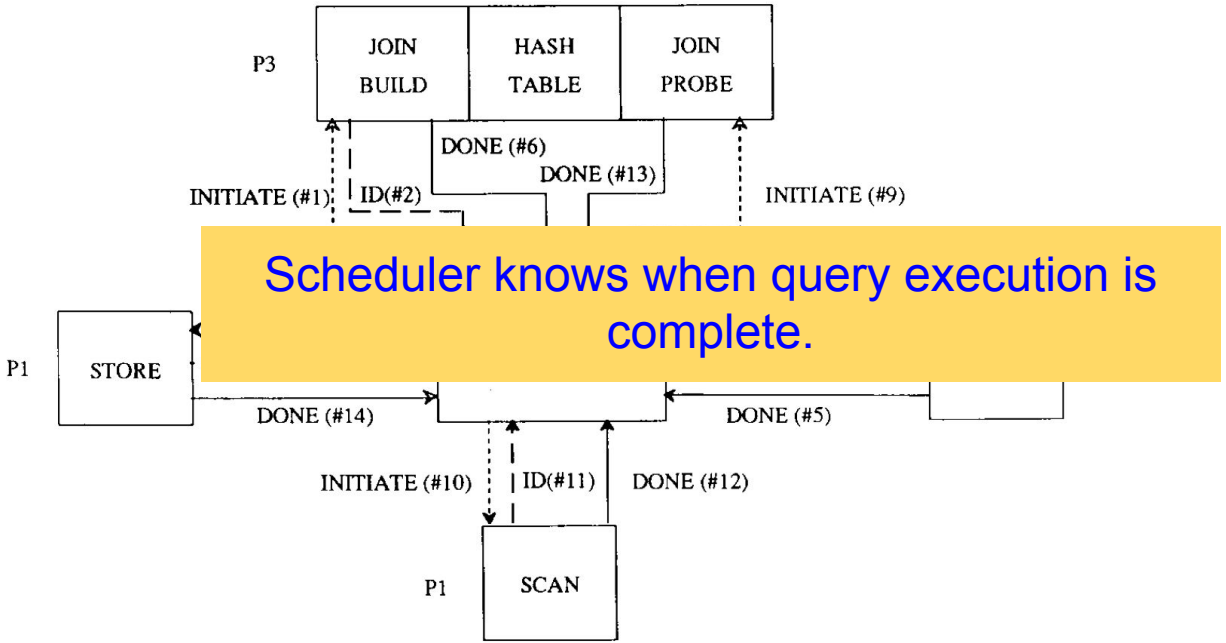


Fig. 5.

Scheduler Terminates Query





Scheduler knows when query execution is complete.

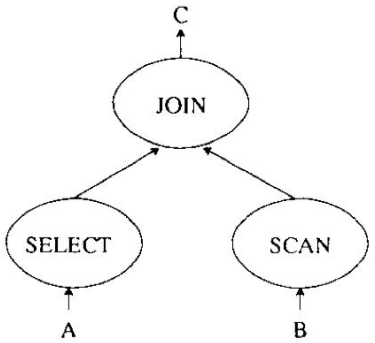
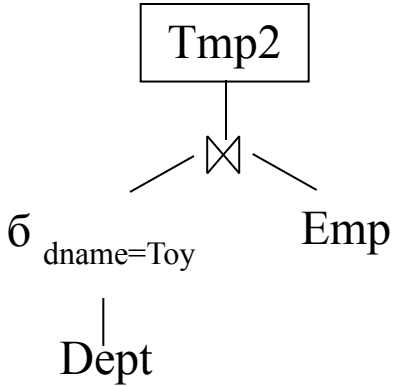
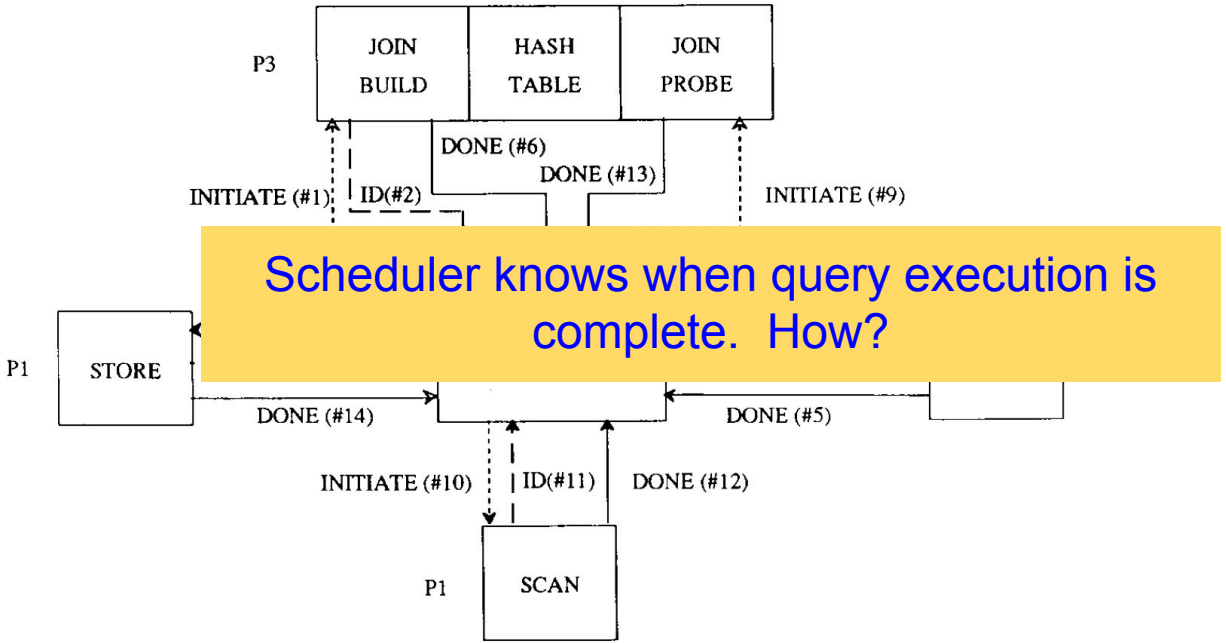


Fig. 5.

Scheduler Coordinates Flow of Data





Scheduler knows when query execution is complete. How?

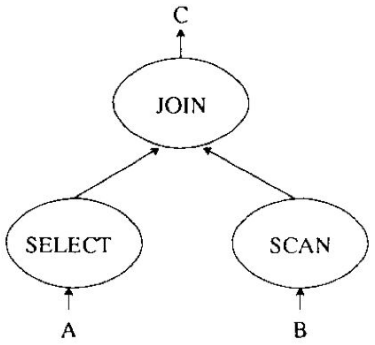
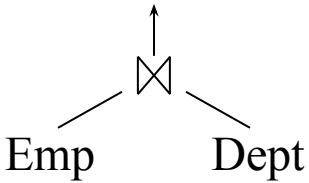


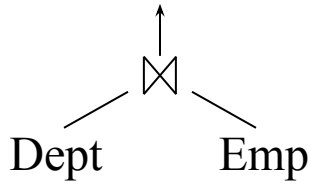
Fig. 5.

Hash-Join: Not Enough Memory



What happens if the table in the build phase does not fit in memory? Example: Emp table is 100 Gigabytes and the amount of memory is 16 Gigabytes.

Hash-Join: Not Enough Memory



What happens if the table in the build phase does not fit in memory? Example: Emp table is 100 Gigabytes and the amount of memory is 16 Gigabytes. Solution:

1. Make sure the inner (build) table is the smaller of the two tables, say Dept.
2. Break the inner table into M buckets using a hash function. The ideal function ensures each bucket is small enough to fit in memory. Write these buckets to disk.
3. Break the outer table (Emp) into M buckets using the same hash function used in Step 2. Write these buckets to disk.
4. For i = 1 to M:
    a. Stage Bucket i of Dept in an in-memory hash table
    b. Read Bucket i of Emp and probe the hash table for joining records

Hash-Join: Not Enough Memory

- Divide-and-conquer: Records in bucket i of the inner table (Dept) may join with the records in bucket i of the outer table (Emp) only.
  - Use of a hash function has reduced 1 join into M small joins.

- The first bucket of the inner table (Dept) does not have to be written to disk, it can be staged in memory. Similarly, the first bucket of the outer table (Emp) is not required to be written to disk. It may probe the hash table of the inner table (Dept) for joining records.

Hash-Join: Multiple Nodes

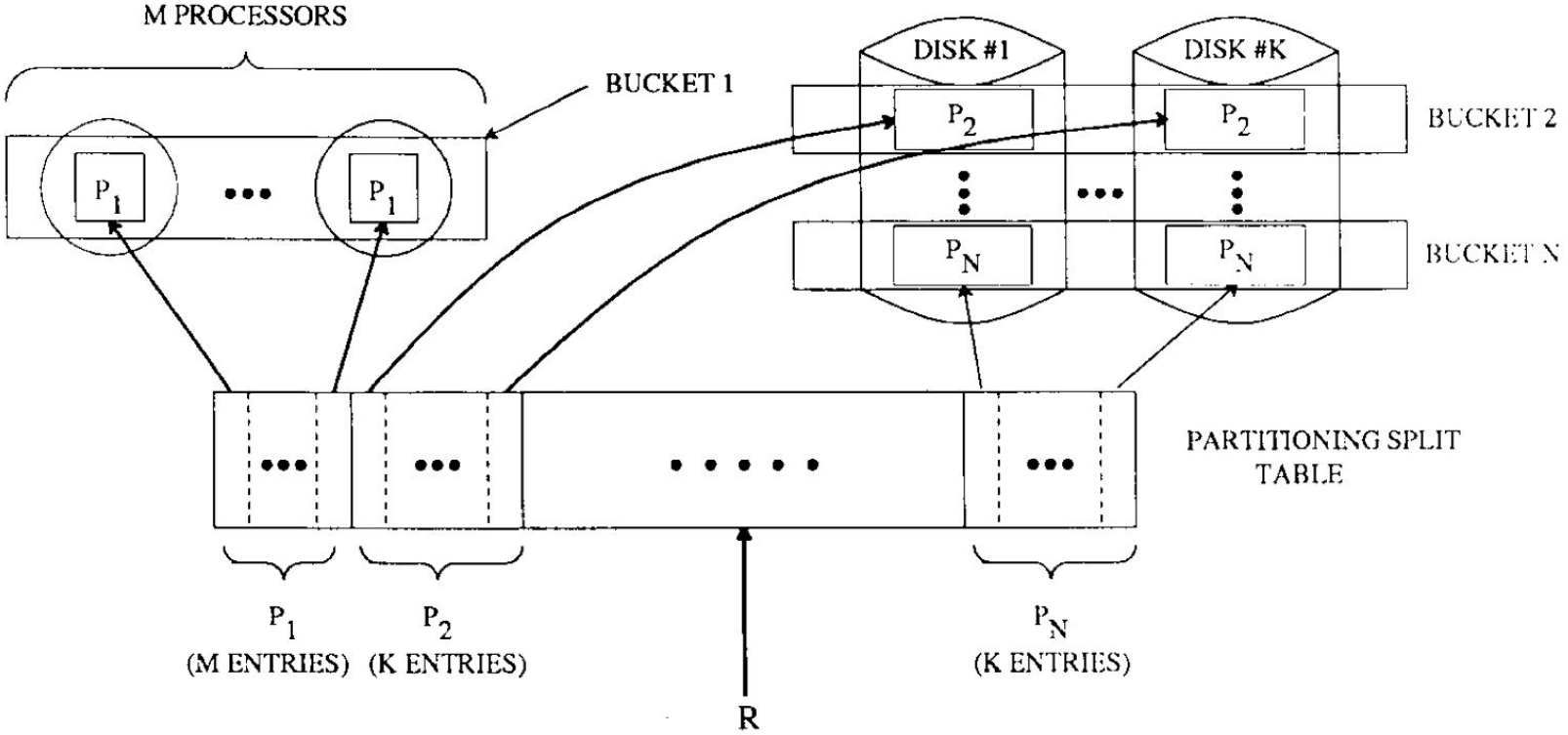


Fig. 8. Partitioning of R into N logical buckets for hybrid hash–join.

Software Modularity: Split Table

- Once a process begins execution, it continuously reads records from its input stream, operates on each tuple, and uses a split table to route the resulting tuple to the process indicated in the split table.
- When the process detects the end of its input stream, it first closes the output streams and then sends a control message to its scheduler process indicating that it has completed execution. Closing the output stream generates an "end of stream" message to each of the destination processes.

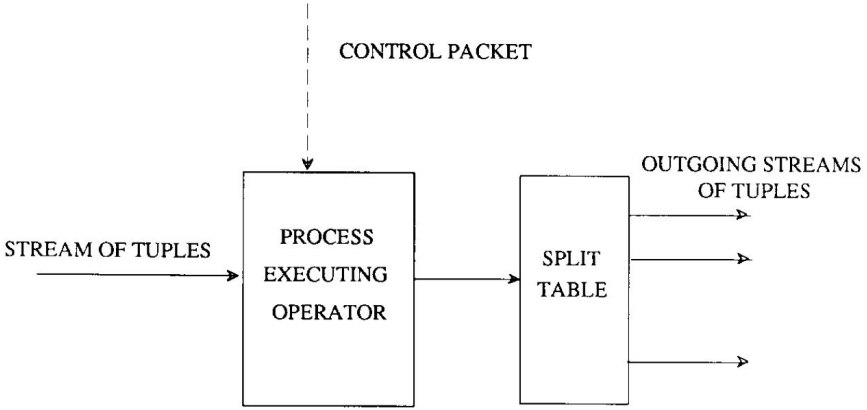


Fig. 3.

Value	Destination Process
0	(Processor #3, Port #5)
1	(Processor #2, Port #13)
2	(Processor #7, Port #6)
3	(Processor #9, Port #15)

Fig. 4. An example split table.

Node Failures: Interleaved Declustering

Maintain data availability by constructing a primary and a backup replica of fragments. How? Interleaved declustering.

Node	Cluster 0				Cluster 1			
	0	1	2	3	4	5	6	7
Primary Copy	R0	R1	R2	R3	R4	R5	R6	R7
Backup Copy		r0.0	r0.1	r0.2		r4.0	r4.1	r4.2
	r1.2		r1.0	r1.1	r5.2		r5.0	r5.1
	r2.1	r2.2		r2.0	r6.1	r6.2		r6.0
	r3.0	r3.1	r3.2		r7.0	r7.1	r7.2	

Fig. 10. Interleaved declustering (cluster size = 4).

Failure of Node 1 distributes its load on Nodes 0, 2 and 3. Why not increase cluster size to 8?

Node Failures: Chained Declustering

Maintain primary and backup replicas of a fragment on adjacent nodes.

Node	0	1	2	3	4	5	6	7
Primary Copy	R0	R1	R2	R3	R4	R5	R6	R7
Backup Copy	r7	r0	r1	r2	r3	r4	r5	r6

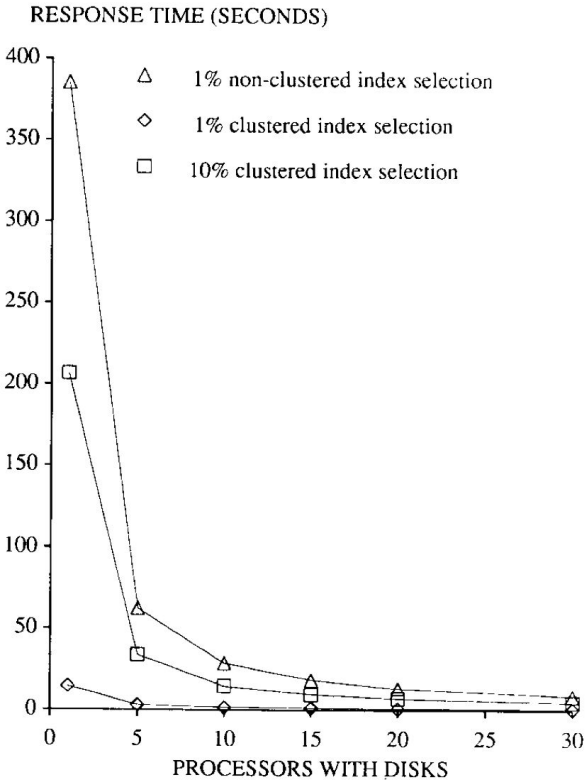
Node	0	1	2	3	4	5	6	7
Primary Copy	R0	---	$\frac{1}{7}$R2	$\frac{2}{7}$R3	$\frac{3}{7}$R4	$\frac{4}{7}$R5	$\frac{5}{7}$R6	$\frac{6}{7}$R7
Backup Copy	$\frac{1}{7}$r7	---	r1	$\frac{6}{7}$r2	$\frac{5}{7}$r3	$\frac{4}{7}$r4	$\frac{3}{7}$r5	$\frac{2}{7}$r6

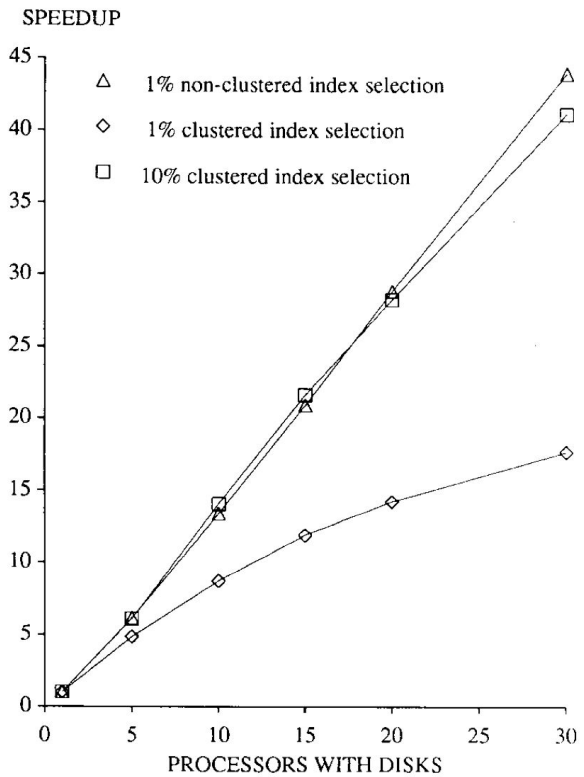
In the presence of failures, push a fraction of the load of a failed node to its adjacent node, balancing the workload across all nodes.

Select From one Table

- How is speedup computed?
- Why is speedup superlinear?

- TMP2 $\leftarrow \sigma_{attr>x}(R)$

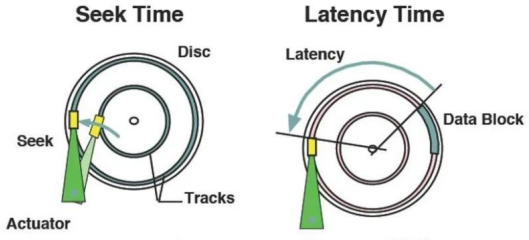




Superlinear Speedup: Multi-Zone Disk Drive

- Zones on the outer-regions of a disk drive provide a significantly higher bandwidth than the inner-regions of the disk drive. Transfer rate of each disk is higher with N disks when compared with 1 disk.
- Seek time is a function of the distance travelled by the disk head. The seek time is higher with one disk when compared with N disks.

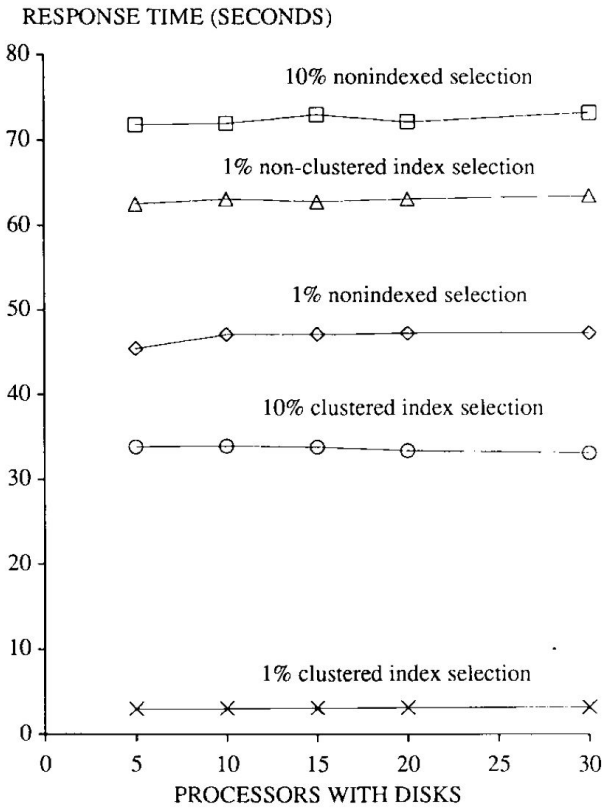
Zone #	Size (MB)	Transfer Rate (MBps)
0	139.7	4.17
1	67.0	4.08
2	45.7	4.02
3	45.3	3.97
4	43.9	3.88
5	42.6	3.83
6	41.3	3.74
7	56.4	3.60
8	39.2	3.60
9	37.5	3.50
10	51.5	3.36
11	35.0	3.31
12	33.8	3.22
13	46.5	3.13
14	31.8	3.07
15	30.7	2.99
16	41.8	2.85
17	28.0	2.76
18	27.6	2.76
19	37.1	2.62
20	25.0	2.53
21	33.5	2.43
22	25.4	2.33



Select Scaleup

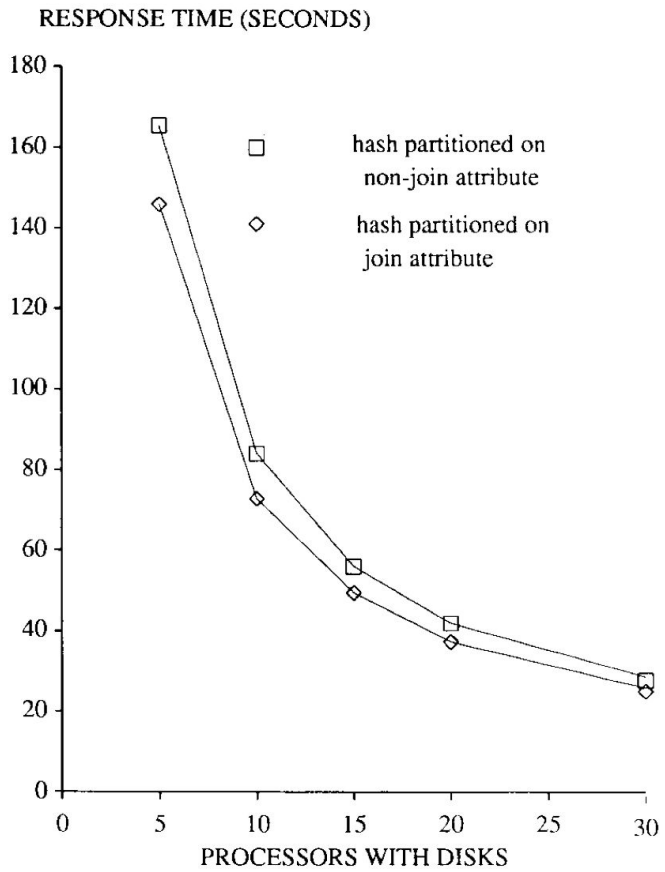
Size of input table increases from 1 Million records to 6 Million records.

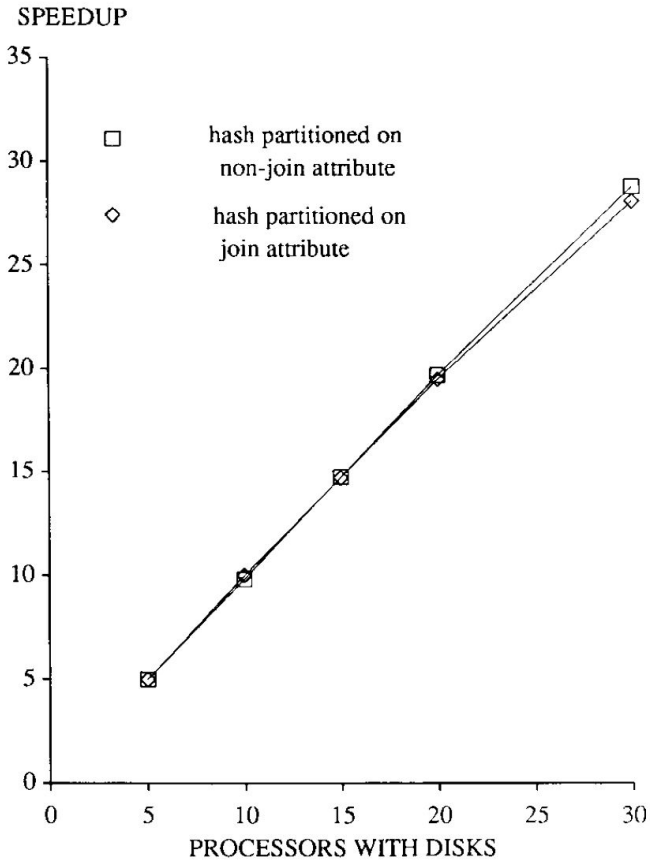
- TMP2 $\leftarrow \sigma_{attr>x}(R)$



Join: Speedup

- TMP2 $\leftarrow A \bowtie B'$





Gamma Contributions

- Alternative declustering strategies including Hybrid Range Partitioning and MAGIC (Next lecture).
- Hybrid hash-join algorithm.
- High availability of data using chained Declustering.
- A data flow execution paradigm.
- A modular and configurable software system using the split operator.
- Use of nodes with disk and nodes without disk.
- Experimental methodology for multi-node DBMSs: Speedup and Scaleup.
- SQL to hide parallelism.